

# From Bias–Variance to Logistic Regression

---

**Tao LIN**

SoE, Westlake University

September 29, 2026



- 1 Review and Continuation
  - Review: regularization and model selection
  - Continue: interpreting bias and variance
- 2 From Regression to Classification
- 3 Logistic Regression: Model, Loss, Update
  - A probability model
  - Learning with cross-entropy
  - Optimization and regularization

# Table of Contents

---

## 1 Review and Continuation

- Review: regularization and model selection
- Continue: interpreting bias and variance

## 2 From Regression to Classification

## 3 Logistic Regression: Model, Loss, Update

# Where we stopped; where we go next

---

**Last time:** we derived the squared-error decomposition.

Expected prediction error = noise + squared bias + variance.

**Today:**

# Where we stopped; where we go next

---

**Last time:** we derived the squared-error decomposition.

Expected prediction error = noise + squared bias + variance.

**Today:**

- 1 Interpret the three terms and connect them to model choice.

# Where we stopped; where we go next

---

**Last time:** we derived the squared-error decomposition.

Expected prediction error = noise + squared bias + variance.

**Today:**

- 1 Interpret the three terms and connect them to model choice.
- 2 Move from real-valued targets to class labels and probabilities.

# Where we stopped; where we go next

---

**Last time:** we derived the squared-error decomposition.

Expected prediction error = noise + squared bias + variance.

**Today:**

- 1 Interpret the three terms and connect them to model choice.
- 2 Move from real-valued targets to class labels and probabilities.
- 3 Build logistic regression: model, loss, and gradient update.

# Where we stopped; where we go next

---

**Last time:** we derived the squared-error decomposition.

Expected prediction error = noise + squared bias + variance.

**Today:**

- 1 Interpret the three terms and connect them to model choice.
- 2 Move from real-valued targets to class labels and probabilities.
- 3 Build logistic regression: model, loss, and gradient update.

Guiding question: How can a linear score produce a useful classifier?

# Table of Contents

---

- 1 Review and Continuation
  - Review: regularization and model selection
  - Continue: interpreting bias and variance
- 2 From Regression to Classification
- 3 Logistic Regression: Model, Loss, Update
  - A probability model
  - Learning with cross-entropy
  - Optimization and regularization

# Review roadmap: four ideas, one decision

---

**Goal:** choose a predictor that works on new data.

---

| Idea                  | Question it answers                                  |
|-----------------------|------------------------------------------------------|
| <b>Regularization</b> | How do we fit candidates with different flexibility? |

---

# Review roadmap: four ideas, one decision

---

**Goal:** choose a predictor that works on new data.

---

| Idea                  | Question it answers                                  |
|-----------------------|------------------------------------------------------|
| <b>Regularization</b> | How do we fit candidates with different flexibility? |
| <b>Bias–variance</b>  | Why can stronger regularization help or hurt?        |

---

# Review roadmap: four ideas, one decision

---

**Goal:** choose a predictor that works on new data.

---

| Idea                      | Question it answers                                  |
|---------------------------|------------------------------------------------------|
| <b>Regularization</b>     | How do we fit candidates with different flexibility? |
| <b>Bias–variance</b>      | Why can stronger regularization help or hurt?        |
| <b>Generalization gap</b> | How reliable is a held-out error estimate?           |

---

# Review roadmap: four ideas, one decision

**Goal:** choose a predictor that works on new data.

| Idea                      | Question it answers                                  |
|---------------------------|------------------------------------------------------|
| <b>Regularization</b>     | How do we fit candidates with different flexibility? |
| <b>Bias–variance</b>      | Why can stronger regularization help or hurt?        |
| <b>Generalization gap</b> | How reliable is a held-out error estimate?           |
| <b>Model selection</b>    | Which candidate should we keep?                      |

Fit candidates  $\longrightarrow$  compare validation errors  $\longrightarrow$  choose  $\lambda$   $\longrightarrow$  test once.

## Review: what is learned, and what is chosen?

---

With a fixed feature map  $\phi$ , ridge regression solves

$$(\hat{\mathbf{w}}_\lambda, \hat{b}_\lambda) \in \arg \min_{\mathbf{w}, b} \left\{ \frac{1}{N} \sum_{n=1}^N (y_n - \phi(\mathbf{x}_n)^\top \mathbf{w} - b)^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

## Review: what is learned, and what is chosen?

---

With a fixed feature map  $\phi$ , ridge regression solves

$$(\hat{\mathbf{w}}_\lambda, \hat{b}_\lambda) \in \arg \min_{\mathbf{w}, b} \left\{ \frac{1}{N} \sum_{n=1}^N (y_n - \phi(\mathbf{x}_n)^\top \mathbf{w} - b)^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

- **Parameters**  $(\mathbf{w}, b)$  are fitted using training data.

## Review: what is learned, and what is chosen?

---

With a fixed feature map  $\phi$ , ridge regression solves

$$(\hat{\mathbf{w}}_\lambda, \hat{b}_\lambda) \in \arg \min_{\mathbf{w}, b} \left\{ \frac{1}{N} \sum_{n=1}^N (y_n - \phi(\mathbf{x}_n)^\top \mathbf{w} - b)^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

- **Parameters**  $(\mathbf{w}, b)$  are fitted using training data.
- **Hyperparameters**, such as polynomial degree and  $\lambda$ , control flexibility.

## Review: what is learned, and what is chosen?

---

With a fixed feature map  $\phi$ , ridge regression solves

$$(\hat{\mathbf{w}}_\lambda, \hat{b}_\lambda) \in \arg \min_{\mathbf{w}, b} \left\{ \frac{1}{N} \sum_{n=1}^N (y_n - \phi(\mathbf{x}_n)^\top \mathbf{w} - b)^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

- **Parameters**  $(\mathbf{w}, b)$  are fitted using training data.
- **Hyperparameters**, such as polynomial degree and  $\lambda$ , control flexibility.
- Increasing  $\lambda$  discourages large coefficients;  $\lambda = 0$  recovers least squares.

## Review: what is learned, and what is chosen?

---

With a fixed feature map  $\phi$ , ridge regression solves

$$(\hat{\mathbf{w}}_\lambda, \hat{b}_\lambda) \in \arg \min_{\mathbf{w}, b} \left\{ \frac{1}{N} \sum_{n=1}^N (y_n - \phi(\mathbf{x}_n)^\top \mathbf{w} - b)^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

- **Parameters**  $(\mathbf{w}, b)$  are fitted using training data.
- **Hyperparameters**, such as polynomial degree and  $\lambda$ , control flexibility.
- Increasing  $\lambda$  discourages large coefficients;  $\lambda = 0$  recovers least squares.

Recall question: why not choose  $\lambda$  by the smallest training squared error?

## Review: what is learned, and what is chosen?

With a fixed feature map  $\phi$ , ridge regression solves

$$(\hat{\mathbf{w}}_\lambda, \hat{b}_\lambda) \in \arg \min_{\mathbf{w}, b} \left\{ \frac{1}{N} \sum_{n=1}^N (y_n - \phi(\mathbf{x}_n)^\top \mathbf{w} - b)^2 + \lambda \|\mathbf{w}\|_2^2 \right\}.$$

- **Parameters**  $(\mathbf{w}, b)$  are fitted using training data.
- **Hyperparameters**, such as polynomial degree and  $\lambda$ , control flexibility.
- Increasing  $\lambda$  discourages large coefficients;  $\lambda = 0$  recovers least squares.

Recall question: why not choose  $\lambda$  by the smallest training squared error?  
Training fit alone does not measure how well the model predicts new data.

## Step by step: a chosen lambda gives a ridge fit

---

For this derivation, center the training features and targets to remove the intercept.

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

1. Scale by 1/2.

## Step by step: a chosen lambda gives a ridge fit

---

For this derivation, center the training features and targets to remove the intercept.

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

1. Scale by 1/2.

$$\nabla J_{\lambda}(\mathbf{w}) = \frac{1}{N} X^{\top} (X\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}$$

2. Differentiate.

## Step by step: a chosen lambda gives a ridge fit

---

For this derivation, center the training features and targets to remove the intercept.

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

1. Scale by 1/2.

$$\nabla J_{\lambda}(\mathbf{w}) = \frac{1}{N} X^{\top} (X\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}$$

2. Differentiate.

$$(X^{\top} X + N\lambda I) \hat{\mathbf{w}}_{\lambda} = X^{\top} \mathbf{y}$$

3. Set to zero.

For  $\lambda > 0$ , the matrix is positive definite: solve this linear system. Recover  $\hat{b}_{\lambda} = \bar{y} - \bar{\mathbf{x}}^{\top} \hat{\mathbf{w}}_{\lambda}$ .

## Step by step: a chosen lambda gives a ridge fit

For this derivation, center the training features and targets to remove the intercept.

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

1. Scale by 1/2.

$$\nabla J_{\lambda}(\mathbf{w}) = \frac{1}{N} X^{\top} (X\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}$$

2. Differentiate.

$$(X^{\top} X + N\lambda I) \hat{\mathbf{w}}_{\lambda} = X^{\top} \mathbf{y}$$

3. Set to zero.

For  $\lambda > 0$ , the matrix is positive definite: solve this linear system. Recover  $\hat{b}_{\lambda} = \bar{y} - \bar{\mathbf{x}}^{\top} \hat{\mathbf{w}}_{\lambda}$ .

This computes  $\hat{\mathbf{w}}_{\lambda}$  for a **given**  $\lambda$ . It does not choose  $\lambda$  for new data.

## Review: population risk versus empirical error

---

Samples  $S = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$  are i.i.d. from an unknown  $\mathcal{D}$ ; a learning algorithm returns  $f_S = \mathcal{A}(S)$ .

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(Y, f(X))]$$

population risk,

$$L_T(f) = \frac{1}{|T|} \sum_{(\mathbf{x}, y) \in T} \ell(y, f(\mathbf{x}))$$

empirical error on  $T$ ,

$$L_S(f_S)$$

training error.

## Review: population risk versus empirical error

---

Samples  $S = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$  are i.i.d. from an unknown  $\mathcal{D}$ ; a learning algorithm returns  $f_S = \mathcal{A}(S)$ .

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(Y, f(X))]$$

population risk,

$$L_T(f) = \frac{1}{|T|} \sum_{(\mathbf{x}, y) \in T} \ell(y, f(\mathbf{x}))$$

empirical error on  $T$ ,

$$L_S(f_S)$$

training error.

- We cannot compute population risk exactly without knowing  $\mathcal{D}$ .

## Review: population risk versus empirical error

---

Samples  $S = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$  are i.i.d. from an unknown  $\mathcal{D}$ ; a learning algorithm returns  $f_S = \mathcal{A}(S)$ .

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(Y, f(X))]$$

population risk,

$$L_T(f) = \frac{1}{|T|} \sum_{(\mathbf{x}, y) \in T} \ell(y, f(\mathbf{x}))$$

empirical error on  $T$ ,

$$L_S(f_S)$$

training error.

- We cannot compute population risk exactly without knowing  $\mathcal{D}$ .
- Independent evaluation data let us estimate it.

## Review: population risk versus empirical error

---

Samples  $S = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$  are i.i.d. from an unknown  $\mathcal{D}$ ; a learning algorithm returns  $f_S = \mathcal{A}(S)$ .

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(Y, f(X))]$$

population risk,

$$L_T(f) = \frac{1}{|T|} \sum_{(\mathbf{x}, y) \in T} \ell(y, f(\mathbf{x}))$$

empirical error on  $T$ ,

$$L_S(f_S)$$

training error.

- We cannot compute population risk exactly without knowing  $\mathcal{D}$ .
- Independent evaluation data let us estimate it.
- Training error reuses the data that selected  $f_S$ , so it can be optimistic.

## Review: three data roles, three different questions

---

| Data     | Question              | Action                    |
|----------|-----------------------|---------------------------|
| Training | Which parameters fit? | Learn ( $\mathbf{w}, b$ ) |

---

## Review: three data roles, three different questions

---

| Data       | Question                            | Action                     |
|------------|-------------------------------------|----------------------------|
| Training   | Which parameters fit?               | Learn ( $\mathbf{w}, b$ )  |
| Validation | Which candidate generalizes better? | Choose degree or $\lambda$ |

---

## Review: three data roles, three different questions

---

| Data       | Question                                | Action                     |
|------------|-----------------------------------------|----------------------------|
| Training   | Which parameters fit?                   | Learn ( $\mathbf{w}, b$ )  |
| Validation | Which candidate generalizes better?     | Choose degree or $\lambda$ |
| Test       | How well does the final choice perform? | Report final performance   |

## Review: three data roles, three different questions

| Data       | Question                                | Action                     |
|------------|-----------------------------------------|----------------------------|
| Training   | Which parameters fit?                   | Learn ( $\mathbf{w}, b$ )  |
| Validation | Which candidate generalizes better?     | Choose degree or $\lambda$ |
| Test       | How well does the final choice perform? | Report final performance   |

Recall question: if we compare 20 values of  $\lambda$  on a “test” set and choose the best, what role has that set played?

## Review: three data roles, three different questions

| Data       | Question                                | Action                     |
|------------|-----------------------------------------|----------------------------|
| Training   | Which parameters fit?                   | Learn ( $\mathbf{w}, b$ )  |
| Validation | Which candidate generalizes better?     | Choose degree or $\lambda$ |
| Test       | How well does the final choice perform? | Report final performance   |

Recall question: if we compare 20 values of  $\lambda$  on a “test” set and choose the best, what role has that set played?

**Validation.** We still need untouched data for the final evaluation.

## Review: three data roles, three different questions

| Data       | Question                                | Action                     |
|------------|-----------------------------------------|----------------------------|
| Training   | Which parameters fit?                   | Learn ( $\mathbf{w}, b$ )  |
| Validation | Which candidate generalizes better?     | Choose degree or $\lambda$ |
| Test       | How well does the final choice perform? | Report final performance   |

Recall question: if we compare 20 values of  $\lambda$  on a “test” set and choose the best, what role has that set played?

**Validation.** We still need untouched data for the final evaluation.

Fit preprocessing on training data too: feature scaling must not use the held-out data.

## Review: why an independent error estimate is useful

---

For a predictor  $f$  fixed independently of an i.i.d. evaluation set  $T$ , and bounded loss  $\ell \in [a, b]$ , with probability at least  $1 - \delta$ ,

$$|L_T(f) - L_{\mathcal{D}}(f)| \leq (b - a) \sqrt{\frac{\log(2/\delta)}{2|T|}}.$$

## Review: why an independent error estimate is useful

---

For a predictor  $f$  fixed independently of an i.i.d. evaluation set  $T$ , and bounded loss  $\ell \in [a, b]$ , with probability at least  $1 - \delta$ ,

$$|L_T(f) - L_{\mathcal{D}}(f)| \leq (b - a) \sqrt{\frac{\log(2/\delta)}{2|T|}}.$$

- More evaluation data give a tighter estimate: the rate is  $1/\sqrt{|T|}$ .

## Review: why an independent error estimate is useful

---

For a predictor  $f$  fixed independently of an i.i.d. evaluation set  $T$ , and bounded loss  $\ell \in [a, b]$ , with probability at least  $1 - \delta$ ,

$$|L_T(f) - L_{\mathcal{D}}(f)| \leq (b - a) \sqrt{\frac{\log(2/\delta)}{2|T|}}.$$

- More evaluation data give a tighter estimate: the rate is  $1/\sqrt{|T|}$ .
- For  $K$  candidates fixed independently of  $T$ , replace  $\log(2/\delta)$  with  $\log(2K/\delta)$  to control all  $K$  deviations at once.

## Review: why an independent error estimate is useful

---

For a predictor  $f$  fixed independently of an i.i.d. evaluation set  $T$ , and bounded loss  $\ell \in [a, b]$ , with probability at least  $1 - \delta$ ,

$$|L_T(f) - L_{\mathcal{D}}(f)| \leq (b - a) \sqrt{\frac{\log(2/\delta)}{2|T|}}.$$

- More evaluation data give a tighter estimate: the rate is  $1/\sqrt{|T|}$ .
- For  $K$  candidates fixed independently of  $T$ , replace  $\log(2/\delta)$  with  $\log(2K/\delta)$  to control all  $K$  deviations at once.
- Selection can exploit validation noise; repeated tuning does not make validation a fresh test set.

## Review: why an independent error estimate is useful

For a predictor  $f$  fixed independently of an i.i.d. evaluation set  $T$ , and bounded loss  $\ell \in [a, b]$ , with probability at least  $1 - \delta$ ,

$$|L_T(f) - L_{\mathcal{D}}(f)| \leq (b - a) \sqrt{\frac{\log(2/\delta)}{2|T|}}.$$

- More evaluation data give a tighter estimate: the rate is  $1/\sqrt{|T|}$ .
- For  $K$  candidates fixed independently of  $T$ , replace  $\log(2/\delta)$  with  $\log(2K/\delta)$  to control all  $K$  deviations at once.
- Selection can exploit validation noise; repeated tuning does not make validation a fresh test set.

Assumptions matter: unbounded squared loss does not automatically satisfy this bound.

## Read the bound: how much validation data?

A fixed classifier, independent i.i.d. validation data, and 0–1 loss:

$$\epsilon(m, \delta) = \sqrt{\frac{\log(2/\delta)}{2m}}, \quad m = |T|, \quad \delta = 0.05.$$

---

| Validation size $m$ | Bound $\epsilon$ |
|---------------------|------------------|
|---------------------|------------------|

---

## Read the bound: how much validation data?

A fixed classifier, independent i.i.d. validation data, and 0–1 loss:

$$\epsilon(m, \delta) = \sqrt{\frac{\log(2/\delta)}{2m}}, \quad m = |T|, \quad \delta = 0.05.$$

|            | Validation size $m$ | Bound $\epsilon$ |
|------------|---------------------|------------------|
| Start here | 200                 | 0.096            |

At  $m = 200$ , a validation error of 0.10 gives the conservative interval  $[0.004, 0.196]$  for population error (rounded).

## Read the bound: how much validation data?

A fixed classifier, independent i.i.d. validation data, and 0–1 loss:

$$\epsilon(m, \delta) = \sqrt{\frac{\log(2/\delta)}{2m}}, \quad m = |T|, \quad \delta = 0.05.$$

|                     | Validation size $m$ | Bound $\epsilon$ |
|---------------------|---------------------|------------------|
| Start here          | 200                 | 0.096            |
| Four times the data | 800                 | 0.048            |

At  $m = 200$ , a validation error of 0.10 gives the conservative interval  $[0.004, 0.196]$  for population error (rounded).

## Read the bound: how much validation data?

A fixed classifier, independent i.i.d. validation data, and 0–1 loss:

$$\epsilon(m, \delta) = \sqrt{\frac{\log(2/\delta)}{2m}}, \quad m = |T|, \quad \delta = 0.05.$$

|                     | Validation size $m$ | Bound $\epsilon$ |
|---------------------|---------------------|------------------|
| Start here          | 200                 | 0.096            |
| Four times the data | 800                 | 0.048            |

At  $m = 200$ , a validation error of 0.10 gives the conservative interval  $[0.004, 0.196]$  for population error (rounded).

Four times the data halves the bound; higher confidence makes it wider. This is a probability guarantee, not the gap of every observed sample.

## Step by step: compare candidates on the same data

Illustrative ridge results: same features, training set, and validation set. All errors below are **mean squared error without the penalty**.

| Chosen $\lambda$ | Training MSE |
|------------------|--------------|
| 0                | 0.04         |
| 0.1              | 0.07         |
| 10               | 0.22         |

**1. Fit:** training error alone favors  $\lambda = 0$ . Should we choose it?

## Step by step: compare candidates on the same data

Illustrative ridge results: same features, training set, and validation set. All errors below are **mean squared error without the penalty**.

| Chosen $\lambda$ | Training MSE | Validation MSE |
|------------------|--------------|----------------|
| 0                | 0.04         | 0.30           |
| 0.1              | 0.07         | 0.12           |
| 10               | 0.22         | 0.25           |

**2. Compare:** which candidate predicts the held-out responses best?

## Step by step: compare candidates on the same data

Illustrative ridge results: same features, training set, and validation set. All errors below are **mean squared error without the penalty**.

| Chosen $\lambda$ | Training MSE | Validation MSE |
|------------------|--------------|----------------|
| 0                | 0.04         | 0.30           |
| 0.1              | 0.07         | 0.12           |
| 10               | 0.22         | 0.25           |

**3. Select:**  $\hat{\lambda} = 0.1$  minimizes validation MSE among these candidates.

## Step by step: compare candidates on the same data

Illustrative ridge results: same features, training set, and validation set. All errors below are **mean squared error without the penalty**.

| Chosen $\lambda$ | Training MSE | Validation MSE |
|------------------|--------------|----------------|
| 0                | 0.04         | 0.30           |
| 0.1              | 0.07         | 0.12           |
| 10               | 0.22         | 0.25           |

**3. Select:**  $\hat{\lambda} = 0.1$  minimizes validation MSE among these candidates.

**4. Evaluate:** after all model choices, use untouched test data. The selected validation minimum is not a final test result.

## Review: K-fold cross-validation for model selection

---

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.

Red: validation fold

White: training folds

## Review: K-fold cross-validation for model selection

---

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.



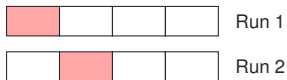
Red: validation fold  
White: training folds

## Review: K-fold cross-validation for model selection

---

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.
- 3 Repeat for every held-out fold and average the validation losses.



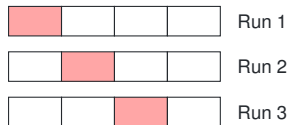
Red: validation fold  
White: training folds

## Review: K-fold cross-validation for model selection

---

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.
- 3 Repeat for every held-out fold and average the validation losses.



Red: validation fold  
White: training folds

## Review: K-fold cross-validation for model selection

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.
- 3 Repeat for every held-out fold and average the validation losses.



## Review: K-fold cross-validation for model selection

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.
- 3 Repeat for every held-out fold and average the validation losses.



- Refit preprocessing inside each fold; reserve a separate final test set.

## Review: K-fold cross-validation for model selection

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.
- 3 Repeat for every held-out fold and average the validation losses.



- Refit preprocessing inside each fold; reserve a separate final test set.
- Choose the setting with the best mean validation performance, then refit on development data.

## Review: K-fold cross-validation for model selection

Use the same folds for every candidate setting:

- 1 Split development data into  $K$  folds.
- 2 Fit on  $K - 1$  folds; validate on the remaining fold.
- 3 Repeat for every held-out fold and average the validation losses.



- Refit preprocessing inside each fold; reserve a separate final test set.
- Choose the setting with the best mean validation performance, then refit on development data.
- CV is useful but is not guaranteed to be an exactly unbiased risk estimate.

## Review checkpoint: connect the choices to the formula

---

- 1 Why can a model with lower training error have higher unseen error?

## Review checkpoint: connect the choices to the formula

---

- 1 Why can a model with lower training error have higher unseen error?
- 2 What does regularization change about the fitted predictor?

## Review checkpoint: connect the choices to the formula

---

- 1 Why can a model with lower training error have higher unseen error?
- 2 What does regularization change about the fitted predictor?
- 3 How do we choose regularization strength without knowing the true function?

## Review checkpoint: connect the choices to the formula

---

- 1 Why can a model with lower training error have higher unseen error?
- 2 What does regularization change about the fitted predictor?
- 3 How do we choose regularization strength without knowing the true function?

Training fit can include sample-specific noise. Regularization changes the fit and its sensitivity to the sample. Validation or cross-validation compares the resulting predictors on held-out data.

**Where we stopped last time:** the bias–variance formula explains how average error depends on systematic error and sample sensitivity.

# Table of Contents

---

## 1 Review and Continuation

- Review: regularization and model selection
- Continue: interpreting bias and variance

## 2 From Regression to Classification

## 3 Logistic Regression: Model, Loss, Update

- A probability model
- Learning with cross-entropy
- Optimization and regularization

## Recall: the bias–variance decomposition

---

Fix an input  $\mathbf{x}_0$ . Write  $Y = f(\mathbf{x}_0) + \epsilon$ , where  $\mathbb{E}[\epsilon \mid \mathbf{x}_0] = 0$  and the fresh noise is independent of training set  $S$ . Let  $\bar{f}(\mathbf{x}_0) = \mathbb{E}_S[f_S(\mathbf{x}_0)]$ .

$$\begin{aligned}\mathbb{E}_{S,\epsilon} [(Y - f_S(\mathbf{x}_0))^2] &= \underbrace{\sigma^2(\mathbf{x}_0)}_{\text{noise}} + \underbrace{(f(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2}_{\text{squared bias}} \\ &\quad + \underbrace{\mathbb{E}_S [(f_S(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2]}_{\text{variance}}.\end{aligned}$$

## Recall: the bias–variance decomposition

---

Fix an input  $\mathbf{x}_0$ . Write  $Y = f(\mathbf{x}_0) + \epsilon$ , where  $\mathbb{E}[\epsilon \mid \mathbf{x}_0] = 0$  and the fresh noise is independent of training set  $S$ . Let  $\bar{f}(\mathbf{x}_0) = \mathbb{E}_S[f_S(\mathbf{x}_0)]$ .

$$\begin{aligned}\mathbb{E}_{S,\epsilon} [(Y - f_S(\mathbf{x}_0))^2] &= \underbrace{\sigma^2(\mathbf{x}_0)}_{\text{noise}} + \underbrace{(f(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2}_{\text{squared bias}} \\ &\quad + \underbrace{\mathbb{E}_S [(f_S(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2]}_{\text{variance}}.\end{aligned}$$

- Average over repeated training sets of the same size, with the same learning procedure.

## Recall: the bias–variance decomposition

---

Fix an input  $\mathbf{x}_0$ . Write  $Y = f(\mathbf{x}_0) + \epsilon$ , where  $\mathbb{E}[\epsilon \mid \mathbf{x}_0] = 0$  and the fresh noise is independent of training set  $S$ . Let  $\bar{f}(\mathbf{x}_0) = \mathbb{E}_S[f_S(\mathbf{x}_0)]$ .

$$\begin{aligned}\mathbb{E}_{S,\epsilon} [(Y - f_S(\mathbf{x}_0))^2] &= \underbrace{\sigma^2(\mathbf{x}_0)}_{\text{noise}} + \underbrace{(f(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2}_{\text{squared bias}} \\ &\quad + \underbrace{\mathbb{E}_S [(f_S(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2]}_{\text{variance}}.\end{aligned}$$

- Average over repeated training sets of the same size, with the same learning procedure.
- Keep  $\mathbf{x}_0$  fixed: variance concerns predictions across training sets.

## Recall: the bias–variance decomposition

---

Fix an input  $\mathbf{x}_0$ . Write  $Y = f(\mathbf{x}_0) + \epsilon$ , where  $\mathbb{E}[\epsilon \mid \mathbf{x}_0] = 0$  and the fresh noise is independent of training set  $S$ . Let  $\bar{f}(\mathbf{x}_0) = \mathbb{E}_S[f_S(\mathbf{x}_0)]$ .

$$\begin{aligned}\mathbb{E}_{S,\epsilon} [(Y - f_S(\mathbf{x}_0))^2] &= \underbrace{\sigma^2(\mathbf{x}_0)}_{\text{noise}} + \underbrace{(f(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2}_{\text{squared bias}} \\ &\quad + \underbrace{\mathbb{E}_S [(f_S(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2]}_{\text{variance}}.\end{aligned}$$

- Average over repeated training sets of the same size, with the same learning procedure.
- Keep  $\mathbf{x}_0$  fixed: variance concerns predictions across training sets.
- This is an **equality**; only the noise term is an irreducible lower bound.

## Recall: the bias–variance decomposition

---

Fix an input  $\mathbf{x}_0$ . Write  $Y = f(\mathbf{x}_0) + \epsilon$ , where  $\mathbb{E}[\epsilon \mid \mathbf{x}_0] = 0$  and the fresh noise is independent of training set  $S$ . Let  $\bar{f}(\mathbf{x}_0) = \mathbb{E}_S[f_S(\mathbf{x}_0)]$ .

$$\begin{aligned}\mathbb{E}_{S,\epsilon} [(Y - f_S(\mathbf{x}_0))^2] &= \underbrace{\sigma^2(\mathbf{x}_0)}_{\text{noise}} + \underbrace{(f(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2}_{\text{squared bias}} \\ &\quad + \underbrace{\mathbb{E}_S [(f_S(\mathbf{x}_0) - \bar{f}(\mathbf{x}_0))^2]}_{\text{variance}}.\end{aligned}$$

- Average over repeated training sets of the same size, with the same learning procedure.
- Keep  $\mathbf{x}_0$  fixed: variance concerns predictions across training sets.
- This is an **equality**; only the noise term is an irreducible lower bound.

Averaging this identity over a fresh  $X$  gives the expected population squared error.

## Noise: uncertainty even if we know the true function

---



## Noise: uncertainty even if we know the true function

---



- With prediction  $f(\mathbf{x}_0) = \mathbb{E}[Y \mid \mathbf{x}_0]$ , fresh labels still fluctuate.

## Noise: uncertainty even if we know the true function

---



- With prediction  $f(\mathbf{x}_0) = \mathbb{E}[Y \mid \mathbf{x}_0]$ , fresh labels still fluctuate.
- The remaining squared error is  $\sigma^2(\mathbf{x}_0) = \text{Var}(Y \mid \mathbf{x}_0)$ .

# Noise: uncertainty even if we know the true function

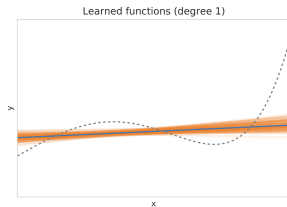
---



- With prediction  $f(\mathbf{x}_0) = \mathbb{E}[Y \mid \mathbf{x}_0]$ , fresh labels still fluctuate.
- The remaining squared error is  $\sigma^2(\mathbf{x}_0) = \text{Var}(Y \mid \mathbf{x}_0)$ .
- A more flexible predictor cannot remove this uncertainty for the given inputs.

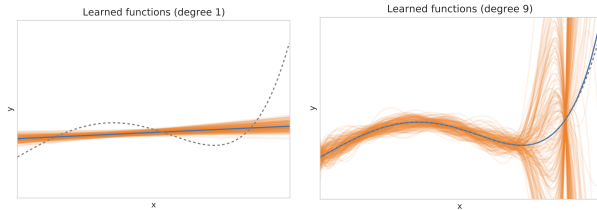
# Squared bias: is the average prediction in the right place?

---

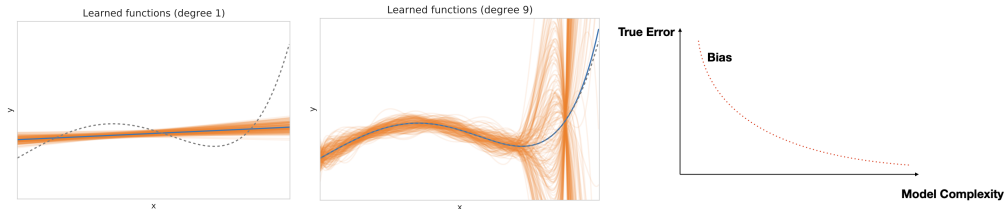


# Squared bias: is the average prediction in the right place?

---

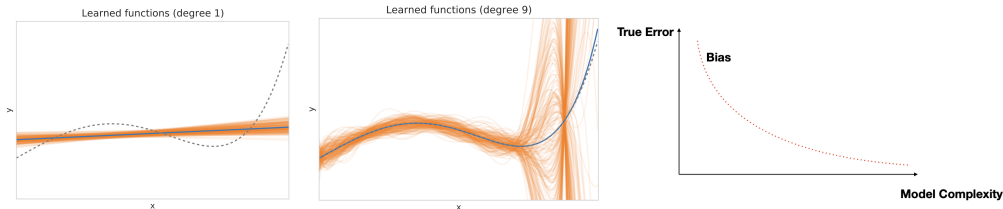


# Squared bias: is the average prediction in the right place?



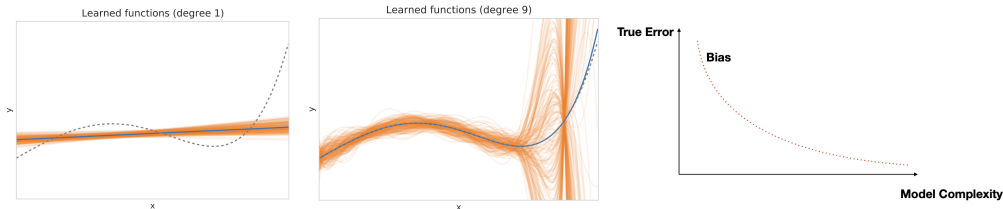
- Compare the average fitted prediction  $\bar{f}(x_0)$  with the true  $f(x_0)$ .

# Squared bias: is the average prediction in the right place?



- Compare the average fitted prediction  $\bar{f}(x_0)$  with the true  $f(x_0)$ .
- A model that is too rigid can miss the pattern even across many training sets.

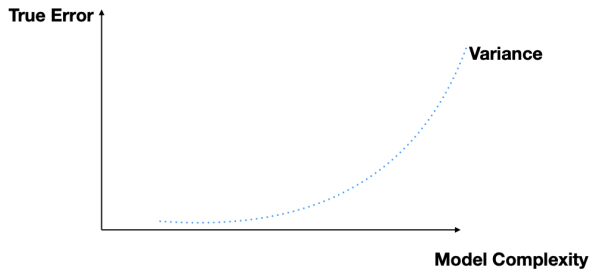
# Squared bias: is the average prediction in the right place?



- Compare the average fitted prediction  $\bar{f}(x_0)$  with the true  $f(x_0)$ .
- A model that is too rigid can miss the pattern even across many training sets.
- In this example, increasing flexibility reduces squared bias.

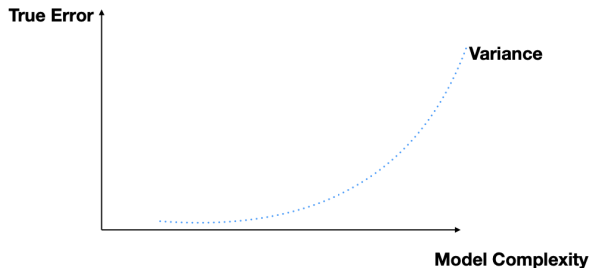
# Variance: would a new training set change the prediction?

---



## Variance: would a new training set change the prediction?

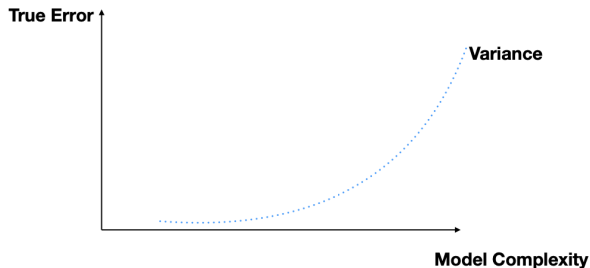
---



- Refit the same learning procedure on different training samples.

## Variance: would a new training set change the prediction?

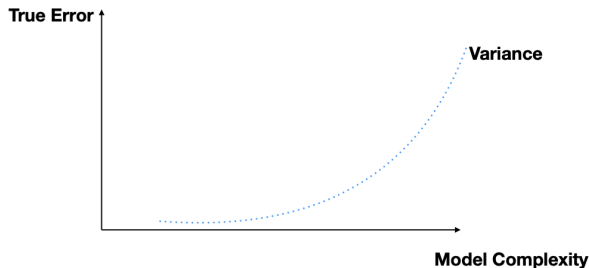
---



- Refit the same learning procedure on different training samples.
- At the **same input**, measure the spread around  $\bar{f}(x_0)$ .

## Variance: would a new training set change the prediction?

---



- Refit the same learning procedure on different training samples.
- At the **same input**, measure the spread around  $\bar{f}(x_0)$ .
- A flexible fit can respond strongly to small changes in the sample.

## A numerical check at one input

Suppose  $f(\mathbf{x}_0) = 2$  and the noise variance is 1. In a toy experiment, each of the three training outcomes is equally likely.

|                                  | Procedure A | Procedure B |
|----------------------------------|-------------|-------------|
| Predictions across training sets | 0, 1, 2     | 0, 2, 4     |

**Think first:** Which procedure has lower bias? Which has lower total error?

## A numerical check at one input

Suppose  $f(\mathbf{x}_0) = 2$  and the noise variance is 1. In a toy experiment, each of the three training outcomes is equally likely.

|                                  | Procedure A | Procedure B |
|----------------------------------|-------------|-------------|
| Predictions across training sets | 0, 1, 2     | 0, 2, 4     |
| Mean prediction                  | 1           | 2           |

## A numerical check at one input

Suppose  $f(\mathbf{x}_0) = 2$  and the noise variance is 1. In a toy experiment, each of the three training outcomes is equally likely.

|                                  | Procedure A     | Procedure B     |
|----------------------------------|-----------------|-----------------|
| Predictions across training sets | 0, 1, 2         | 0, 2, 4         |
| Mean prediction                  | 1               | 2               |
| Squared bias                     | $(2 - 1)^2 = 1$ | $(2 - 2)^2 = 0$ |

## A numerical check at one input

Suppose  $f(\mathbf{x}_0) = 2$  and the noise variance is 1. In a toy experiment, each of the three training outcomes is equally likely.

|                                  | Procedure A     | Procedure B     |
|----------------------------------|-----------------|-----------------|
| Predictions across training sets | 0, 1, 2         | 0, 2, 4         |
| Mean prediction                  | 1               | 2               |
| Squared bias                     | $(2 - 1)^2 = 1$ | $(2 - 2)^2 = 0$ |
| Variance (average over outcomes) | $2/3$           | $8/3$           |

## A numerical check at one input

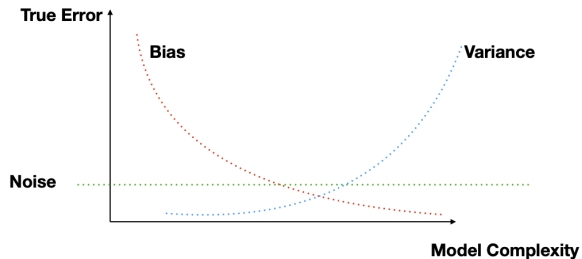
Suppose  $f(\mathbf{x}_0) = 2$  and the noise variance is 1. In a toy experiment, each of the three training outcomes is equally likely.

|                                  | Procedure A         | Procedure B          |
|----------------------------------|---------------------|----------------------|
| Predictions across training sets | 0, 1, 2             | 0, 2, 4              |
| Mean prediction                  | 1                   | 2                    |
| Squared bias                     | $(2 - 1)^2 = 1$     | $(2 - 2)^2 = 0$      |
| Variance (average over outcomes) | $2/3$               | $8/3$                |
| Expected squared error           | $1 + 1 + 2/3 = 8/3$ | $1 + 0 + 8/3 = 11/3$ |

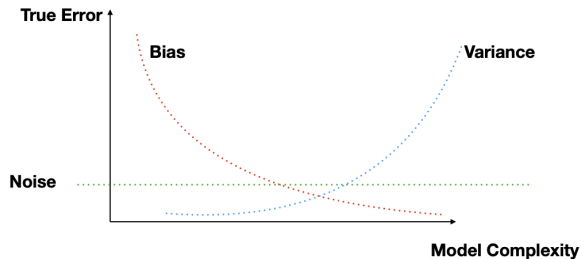
Zero bias does not guarantee the smallest prediction error.

# From the tradeoff to regularization

---

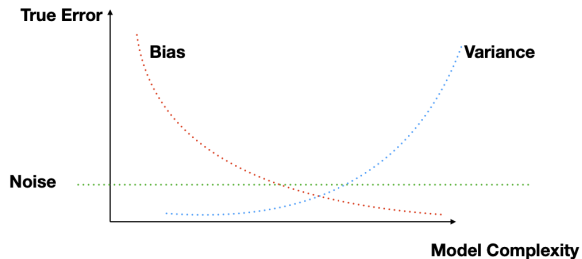


# From the tradeoff to regularization



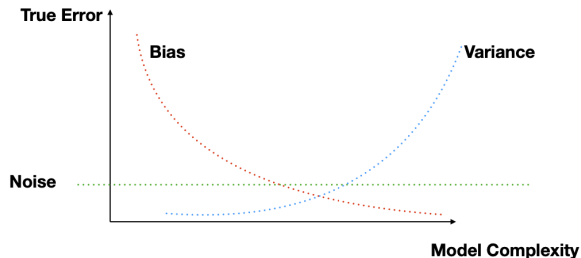
- Too little flexibility can give high bias.

# From the tradeoff to regularization



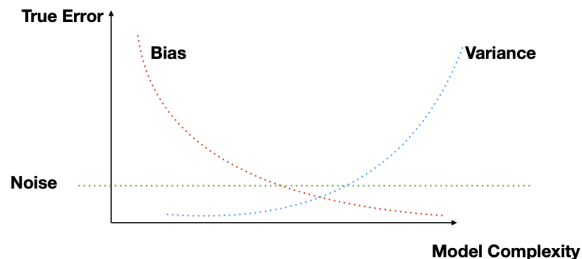
- Too little flexibility can give high bias.
- Too much sensitivity can give high variance.

# From the tradeoff to regularization



- Too little flexibility can give high bias.
- Too much sensitivity can give high variance.
- Stronger ridge regularization typically reduces variance at a cost in bias.

# From the tradeoff to regularization



- Too little flexibility can give high bias.
- Too much sensitivity can give high variance.
- Stronger ridge regularization typically reduces variance at a cost in bias.

The U-shape is a useful classical picture, not a universal law. Choose complexity or  $\lambda$  using validation performance.

# What can we actually measure?

---

We usually have one dataset and do not know  $f$ : we cannot directly read off bias and variance.

# What can we actually measure?

---

We usually have one dataset and do not know  $f$ : we cannot directly read off bias and variance.

- 1 **Train:** fit parameters for each candidate model.

# What can we actually measure?

---

We usually have one dataset and do not know  $f$ : we cannot directly read off bias and variance.

- 1 **Train:** fit parameters for each candidate model.
- 2 **Validate:** choose degree,  $\lambda$ , or other hyperparameters.

# What can we actually measure?

---

We usually have one dataset and do not know  $f$ : we cannot directly read off bias and variance.

- 1 **Train:** fit parameters for each candidate model.
- 2 **Validate:** choose degree,  $\lambda$ , or other hyperparameters.
- 3 **Test:** evaluate the chosen procedure on untouched data.

# What can we actually measure?

---

We usually have one dataset and do not know  $f$ : we cannot directly read off bias and variance.

- 1 **Train:** fit parameters for each candidate model.
- 2 **Validate:** choose degree,  $\lambda$ , or other hyperparameters.
- 3 **Test:** evaluate the chosen procedure on untouched data.

High training and validation error can suggest under-fitting.

Low training error but much higher validation error can suggest over-fitting.

These are diagnostic clues, not direct measurements of bias and variance.

# What can we actually measure?

---

We usually have one dataset and do not know  $f$ : we cannot directly read off bias and variance.

- 1 **Train:** fit parameters for each candidate model.
- 2 **Validate:** choose degree,  $\lambda$ , or other hyperparameters.
- 3 **Test:** evaluate the chosen procedure on untouched data.

High training and validation error can suggest under-fitting.

Low training error but much higher validation error can suggest over-fitting.

These are diagnostic clues, not direct measurements of bias and variance.

Cross-validation repeats the train/validation roles within the development data; the final test set stays separate.

# Table of Contents

---

- 1 Review and Continuation
- 2 From Regression to Classification**
- 3 Logistic Regression: Model, Loss, Update

## What changes when the target is a class?

---

|        | <b>Regression</b>  | <b>Binary classification</b> |
|--------|--------------------|------------------------------|
| Target | $y \in \mathbb{R}$ | $y \in \{0, 1\}$             |

---

## What changes when the target is a class?

---

|         | <b>Regression</b>       | <b>Binary classification</b> |
|---------|-------------------------|------------------------------|
| Target  | $y \in \mathbb{R}$      | $y \in \{0, 1\}$             |
| Example | Amount of a transaction | Fraud or legitimate          |

---

## What changes when the target is a class?

---

|            | <b>Regression</b>       | <b>Binary classification</b>  |
|------------|-------------------------|-------------------------------|
| Target     | $y \in \mathbb{R}$      | $y \in \{0, 1\}$              |
| Example    | Amount of a transaction | Fraud or legitimate           |
| Prediction | Real-valued response    | Class probability, then label |

---

## What changes when the target is a class?

|            | <b>Regression</b>       | <b>Binary classification</b>  |
|------------|-------------------------|-------------------------------|
| Target     | $y \in \mathbb{R}$      | $y \in \{0, 1\}$              |
| Example    | Amount of a transaction | Fraud or legitimate           |
| Prediction | Real-valued response    | Class probability, then label |
| Evaluation | e.g., squared error     | e.g., misclassification rate  |

## What changes when the target is a class?

|            | <b>Regression</b>       | <b>Binary classification</b>  |
|------------|-------------------------|-------------------------------|
| Target     | $y \in \mathbb{R}$      | $y \in \{0, 1\}$              |
| Example    | Amount of a transaction | Fraud or legitimate           |
| Prediction | Real-valued response    | Class probability, then label |
| Evaluation | e.g., squared error     | e.g., misclassification rate  |

Model choice still matters. But the squared-error decomposition above does not transfer unchanged to 0–1 classification error.

We use labels  $\{0, 1\}$  throughout the core lecture.

# Classifier

---

A classifier  $f : \mathcal{X} \rightarrow \mathcal{Y}$

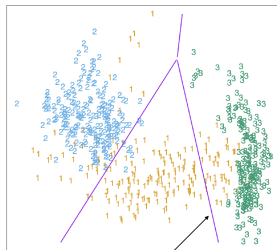
# Classifier

---

A **classifier**  $f : \mathcal{X} \rightarrow \mathcal{Y}$  divides the input space into a collection of regions belonging to each class.

# Classifier

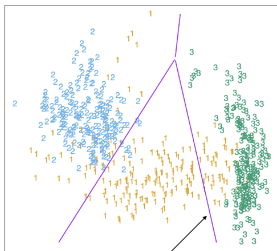
A **classifier**  $f : \mathcal{X} \rightarrow \mathcal{Y}$  divides the input space into a collection of regions belonging to each class.



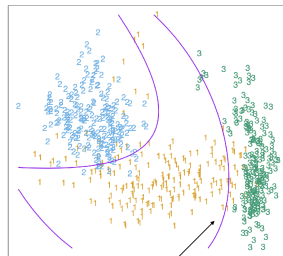
Linear Decision boundary

# Classifier

A **classifier**  $f : \mathcal{X} \rightarrow \mathcal{Y}$  divides the input space into a collection of regions belonging to each class.



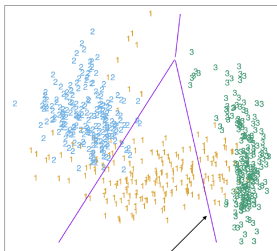
Linear Decision boundary



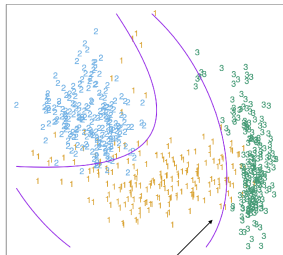
Nonlinear Decision boundary

# Classifier

A **classifier**  $f : \mathcal{X} \rightarrow \mathcal{Y}$  divides the input space into a collection of regions belonging to each class.



Linear Decision boundary



Nonlinear Decision boundary

The boundaries of these regions are called **decision boundaries**.

## Could we regress directly on labels 0 and 1?

---

Yes: fit a least-squares model and threshold its output at 1/2.

$$s(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b, \quad \hat{y} = \mathbf{1}\{s(\mathbf{x}) \geq 1/2\}.$$

## Could we regress directly on labels 0 and 1?

---

Yes: fit a least-squares model and threshold its output at  $1/2$ .

$$s(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b, \quad \hat{y} = \mathbf{1}\{s(\mathbf{x}) \geq 1/2\}.$$

- This is a possible baseline, but  $s(\mathbf{x})$  need not lie in  $[0, 1]$ .

## Could we regress directly on labels 0 and 1?

---

Yes: fit a least-squares model and threshold its output at  $1/2$ .

$$s(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b, \quad \hat{y} = \mathbf{1}\{s(\mathbf{x}) \geq 1/2\}.$$

- This is a possible baseline, but  $s(\mathbf{x})$  need not lie in  $[0, 1]$ .
- Squared loss also penalizes scores far beyond the correct label.

## Could we regress directly on labels 0 and 1?

---

Yes: fit a least-squares model and threshold its output at  $1/2$ .

$$s(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b, \quad \hat{y} = \mathbf{1}\{s(\mathbf{x}) \geq 1/2\}.$$

- This is a possible baseline, but  $s(\mathbf{x})$  need not lie in  $[0, 1]$ .
- Squared loss also penalizes scores far beyond the correct label.
- Extreme inputs can move the fitted line and its decision boundary.

## Could we regress directly on labels 0 and 1?

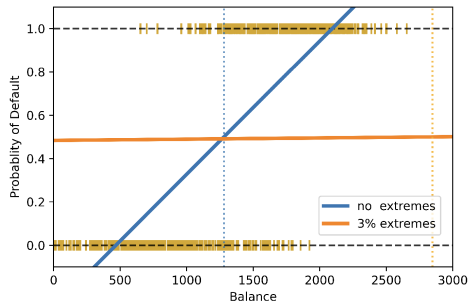
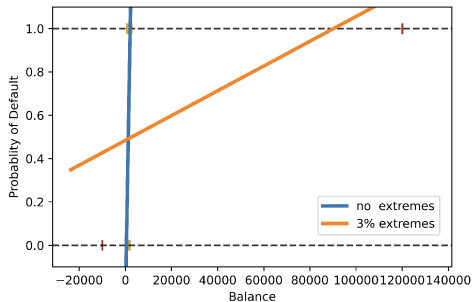
Yes: fit a least-squares model and threshold its output at  $1/2$ .

$$s(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b, \quad \hat{y} = \mathbf{1}\{s(\mathbf{x}) \geq 1/2\}.$$

- This is a possible baseline, but  $s(\mathbf{x})$  need not lie in  $[0, 1]$ .
- Squared loss also penalizes scores far beyond the correct label.
- Extreme inputs can move the fitted line and its decision boundary.

For binary labels,  $\mathbb{E}[Y \mid X = \mathbf{x}] = \Pr(Y = 1 \mid X = \mathbf{x})$ . We want a model that respects this probability range.

# A least-squares boundary can move with extreme inputs



Inspect the picture: does moving an already correctly classified point need to change the classification boundary?

## The ideal decision uses the conditional probability

---

Write  $q(\mathbf{x}) = \Pr(Y = 1 \mid X = \mathbf{x})$ . At this input, which decision makes fewer errors?

| Decision  | When is it wrong? | Conditional error |
|-----------|-------------------|-------------------|
| Predict 0 |                   |                   |
| Predict 1 |                   |                   |

## The ideal decision uses the conditional probability

---

Write  $q(\mathbf{x}) = \Pr(Y = 1 \mid X = \mathbf{x})$ . At this input, which decision makes fewer errors?

| Decision  | When is it wrong?  | Conditional error |
|-----------|--------------------|-------------------|
| Predict 0 | Wrong if $Y = 1$ : | $q(\mathbf{x})$   |
| Predict 1 |                    |                   |

## The ideal decision uses the conditional probability

---

Write  $q(\mathbf{x}) = \Pr(Y = 1 \mid X = \mathbf{x})$ . At this input, which decision makes fewer errors?

| Decision  | When is it wrong? Conditional error    |
|-----------|----------------------------------------|
| Predict 0 | Wrong if $Y = 1$ : $q(\mathbf{x})$     |
| Predict 1 | Wrong if $Y = 0$ : $1 - q(\mathbf{x})$ |

## The ideal decision uses the conditional probability

Write  $q(\mathbf{x}) = \Pr(Y = 1 \mid X = \mathbf{x})$ . At this input, which decision makes fewer errors?

| Decision  | When is it wrong? Conditional error    |
|-----------|----------------------------------------|
| Predict 0 | Wrong if $Y = 1$ : $q(\mathbf{x})$     |
| Predict 1 | Wrong if $Y = 0$ : $1 - q(\mathbf{x})$ |

**Compare the two errors** under equal misclassification costs:

$$1 - q(\mathbf{x}) \leq q(\mathbf{x}) \iff q(\mathbf{x}) \geq \frac{1}{2}, \quad f^*(\mathbf{x}) = \mathbf{1}\{q(\mathbf{x}) \geq \frac{1}{2}\}.$$

- This is the Bayes classifier; either decision is optimal at a tie.

## The ideal decision uses the conditional probability

Write  $q(\mathbf{x}) = \Pr(Y = 1 \mid X = \mathbf{x})$ . At this input, which decision makes fewer errors?

| Decision  | When is it wrong? Conditional error    |
|-----------|----------------------------------------|
| Predict 0 | Wrong if $Y = 1$ : $q(\mathbf{x})$     |
| Predict 1 | Wrong if $Y = 0$ : $1 - q(\mathbf{x})$ |

**Compare the two errors** under equal misclassification costs:

$$1 - q(\mathbf{x}) \leq q(\mathbf{x}) \iff q(\mathbf{x}) \geq \frac{1}{2}, \quad f^*(\mathbf{x}) = \mathbf{1}\{q(\mathbf{x}) \geq \frac{1}{2}\}.$$

- This is the Bayes classifier; either decision is optimal at a tie.
- If  $q(\mathbf{x}) = 0.7$ , predict 1, but the conditional error is still 0.3.

## The ideal decision uses the conditional probability

Write  $q(\mathbf{x}) = \Pr(Y = 1 \mid X = \mathbf{x})$ . At this input, which decision makes fewer errors?

| Decision  | When is it wrong? Conditional error    |
|-----------|----------------------------------------|
| Predict 0 | Wrong if $Y = 1$ : $q(\mathbf{x})$     |
| Predict 1 | Wrong if $Y = 0$ : $1 - q(\mathbf{x})$ |

**Compare the two errors** under equal misclassification costs:

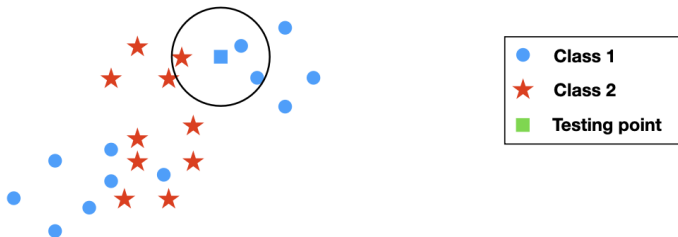
$$1 - q(\mathbf{x}) \leq q(\mathbf{x}) \iff q(\mathbf{x}) \geq \frac{1}{2}, \quad f^*(\mathbf{x}) = \mathbf{1}\{q(\mathbf{x}) \geq \frac{1}{2}\}.$$

- This is the Bayes classifier; either decision is optimal at a tie.
- If  $q(\mathbf{x}) = 0.7$ , predict 1, but the conditional error is still 0.3.
- In practice, we learn an estimate of the unknown  $q$  from data.

# k-Nearest Neighbor

---

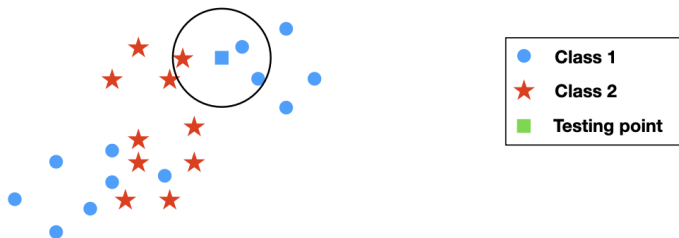
Assume that nearby points are likely to have similar labels



# k-Nearest Neighbor

---

Assume that nearby points are likely to have similar labels



A new point  $x$  is classified based on the **majority vote of its  $k$ -nearest neighbors**.

# Two ways to estimate a class probability

---

**Local averaging (KNN):**

$$\hat{q}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i.$$

## Two ways to estimate a class probability

---

**Local averaging (KNN):**

$$\hat{q}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i.$$

Small  $k$  gives a more variable local fit; larger  $k$  smooths more. Choose  $k$  with validation data.

# Two ways to estimate a class probability

---

## Local averaging (KNN):

$$\hat{q}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i.$$

Small  $k$  gives a more variable local fit; larger  $k$  smooths more. Choose  $k$  with validation data.

## A parametric model (today):

$$p_{\mathbf{w},b}(\mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

Fit a shared set of parameters from the training data.

# Two ways to estimate a class probability

---

## Local averaging (KNN):

$$\hat{q}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i.$$

Small  $k$  gives a more variable local fit; larger  $k$  smooths more. Choose  $k$  with validation data.

## A parametric model (today):

$$p_{\mathbf{w},b}(\mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

Fit a shared set of parameters from the training data.

Both produce a probability estimate; thresholding turns it into a label.

# Table of Contents

---

- 1 Review and Continuation
- 2 From Regression to Classification
- 3 Logistic Regression: Model, Loss, Update**
  - A probability model
  - Learning with cross-entropy
  - Optimization and regularization

# Table of Contents

---

- 1 Review and Continuation
  - Review: regularization and model selection
  - Continue: interpreting bias and variance
- 2 From Regression to Classification
- 3 Logistic Regression: Model, Loss, Update**
  - A probability model
  - Learning with cross-entropy
  - Optimization and regularization

# Turn a linear score into a probability

---

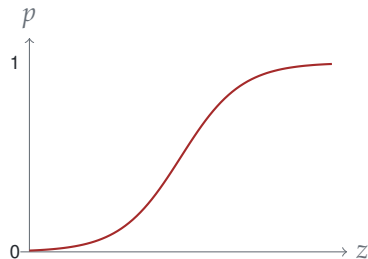
1. **Compute a score:**  $z = \mathbf{x}^\top \mathbf{w} + b$ .

# Turn a linear score into a probability

1. **Compute a score:**  $z = \mathbf{x}^\top \mathbf{w} + b$ .

2. **Map it into  $(0, 1)$ :**

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$



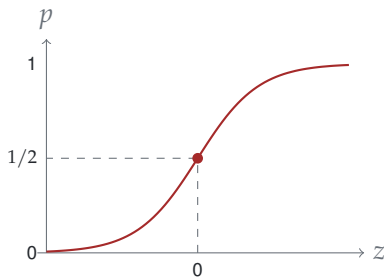
# Turn a linear score into a probability

1. **Compute a score:**  $z = \mathbf{x}^\top \mathbf{w} + b$ .

2. **Map it into  $(0, 1)$ :**

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- $z = 0$  gives  $p = 1/2$ .



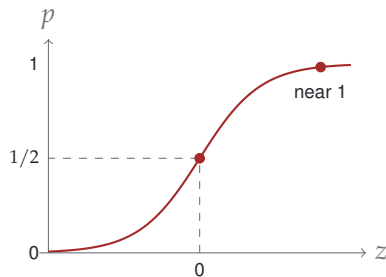
# Turn a linear score into a probability

1. **Compute a score:**  $z = \mathbf{x}^\top \mathbf{w} + b$ .

2. **Map it into  $(0, 1)$ :**

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- $z = 0$  gives  $p = 1/2$ .
- Large positive  $z$ :  $p$  is near 1.



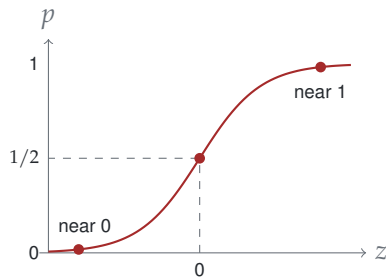
# Turn a linear score into a probability

1. **Compute a score:**  $z = \mathbf{x}^\top \mathbf{w} + b$ .

2. **Map it into  $(0, 1)$ :**

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- $z = 0$  gives  $p = 1/2$ .
- Large positive  $z$ :  $p$  is near 1.
- Large negative  $z$ :  $p$  is near 0.



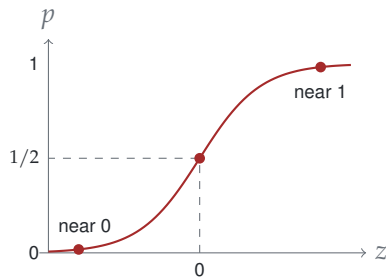
# Turn a linear score into a probability

1. **Compute a score:**  $z = \mathbf{x}^\top \mathbf{w} + b$ .

2. **Map it into  $(0, 1)$ :**

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- $z = 0$  gives  $p = 1/2$ .
- Large positive  $z$ :  $p$  is near 1.
- Large negative  $z$ :  $p$  is near 0.



3. **State the assumption:** linear log-odds,  $\log \frac{p}{1-p} = \mathbf{x}^\top \mathbf{w} + b$ . This restricts the conditional probability model; it is not an identity for every dataset.

# Odds and log-odds: what does a coefficient mean?

1. **Change the scale:** probability  $\rightarrow$  odds  $\rightarrow$  log-odds.

| Probability $p$ | Odds $p/(1 - p)$ | Log-odds |
|-----------------|------------------|----------|
| 0.2             |                  |          |
| 0.5             |                  |          |
| 0.8             |                  |          |

# Odds and log-odds: what does a coefficient mean?

1. **Change the scale:** probability  $\rightarrow$  odds  $\rightarrow$  log-odds.

| Probability $p$ | Odds $p/(1 - p)$ | Log-odds  |
|-----------------|------------------|-----------|
| 0.2             | $1/4$            | $-\log 4$ |
| 0.5             | 1                | 0         |
| 0.8             | 4                | $\log 4$  |

# Odds and log-odds: what does a coefficient mean?

1. **Change the scale:** probability  $\rightarrow$  odds  $\rightarrow$  log-odds.

| Probability $p$ | Odds $p/(1-p)$ | Log-odds  |
|-----------------|----------------|-----------|
| 0.2             | 1/4            | $-\log 4$ |
| 0.5             | 1              | 0         |
| 0.8             | 4              | $\log 4$  |

2. **Increase  $x_j$  by one, holding other features fixed:**

$$z_{\text{new}} = z_{\text{old}} + w_j \implies \text{new odds} = e^{z_{\text{new}}} = e^{w_j} \times \text{old odds}.$$

# Odds and log-odds: what does a coefficient mean?

1. **Change the scale:** probability  $\rightarrow$  odds  $\rightarrow$  log-odds.

| Probability $p$ | Odds $p/(1 - p)$ | Log-odds  |
|-----------------|------------------|-----------|
| 0.2             | 1/4              | $-\log 4$ |
| 0.5             | 1                | 0         |
| 0.8             | 4                | $\log 4$  |

2. **Increase  $x_j$  by one, holding other features fixed:**

$$z_{\text{new}} = z_{\text{old}} + w_j \implies \text{new odds} = e^{z_{\text{new}}} = e^{w_j} \times \text{old odds}.$$

**3. Convert back:** if  $w_j = \log 2$  and initially  $p = 0.5$ , odds go from 1 to 2. The new probability is  $2/(1 + 2) = 2/3$ , not 1.

# Probability, label, and decision boundary

---

## 1. Predict a probability:

$$p_{\mathbf{w},b}(\mathbf{x}) = \Pr_{\mathbf{w},b}(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

# Probability, label, and decision boundary

---

## 1. Predict a probability:

$$p_{\mathbf{w},b}(\mathbf{x}) = \Pr_{\mathbf{w},b}(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

## 2. Apply the threshold: $\sigma$ is increasing and $\sigma(0) = 1/2$ .

$$\hat{y} = \mathbf{1}\{p_{\mathbf{w},b}(\mathbf{x}) \geq 1/2\} = \mathbf{1}\{\mathbf{x}^\top \mathbf{w} + b \geq 0\}.$$

# Probability, label, and decision boundary

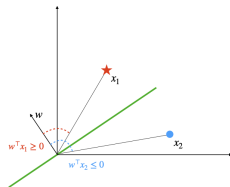
## 1. Predict a probability:

$$p_{\mathbf{w},b}(\mathbf{x}) = \Pr_{\mathbf{w},b}(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

## 2. Apply the threshold: $\sigma$ is increasing and $\sigma(0) = 1/2$ .

$$\hat{y} = \mathbf{1}\{p_{\mathbf{w},b}(\mathbf{x}) \geq 1/2\} = \mathbf{1}\{\mathbf{x}^\top \mathbf{w} + b \geq 0\}.$$

## 3. Locate the boundary: $\mathbf{x}^\top \mathbf{w} + b = 0$ .



# Probability, label, and decision boundary

## 1. Predict a probability:

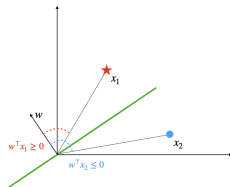
$$p_{\mathbf{w},b}(\mathbf{x}) = \Pr_{\mathbf{w},b}(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

## 2. Apply the threshold: $\sigma$ is increasing and $\sigma(0) = 1/2$ .

$$\hat{y} = \mathbf{1}\{p_{\mathbf{w},b}(\mathbf{x}) \geq 1/2\} = \mathbf{1}\{\mathbf{x}^\top \mathbf{w} + b \geq 0\}.$$

## 3. Locate the boundary: $\mathbf{x}^\top \mathbf{w} + b = 0$ .

- Nonlinear probability; linear boundary.



# Probability, label, and decision boundary

## 1. Predict a probability:

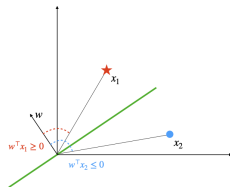
$$p_{\mathbf{w},b}(\mathbf{x}) = \Pr_{\mathbf{w},b}(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \mathbf{w} + b).$$

## 2. Apply the threshold: $\sigma$ is increasing and $\sigma(0) = 1/2$ .

$$\hat{y} = \mathbf{1}\{p_{\mathbf{w},b}(\mathbf{x}) \geq 1/2\} = \mathbf{1}\{\mathbf{x}^\top \mathbf{w} + b \geq 0\}.$$

## 3. Locate the boundary: $\mathbf{x}^\top \mathbf{w} + b = 0$ .

- Nonlinear probability; linear boundary.
- A probability and a predicted label are different outputs.



## Changing the threshold moves the decision boundary

---

**1. Choose class 1 if  $p \geq t$ , where  $0 < t < 1$  and  $p = \sigma(z)$ .**

# Changing the threshold moves the decision boundary

---

**1. Choose class 1 if  $p \geq t$ ,** where  $0 < t < 1$  and  $p = \sigma(z)$ .

$$p \geq t \iff \frac{p}{1-p} \geq \frac{t}{1-t} \quad (\text{odds increase with probability})$$

# Changing the threshold moves the decision boundary

---

**1. Choose class 1 if  $p \geq t$ , where  $0 < t < 1$  and  $p = \sigma(z)$ .**

$$p \geq t \iff \frac{p}{1-p} \geq \frac{t}{1-t} \quad (\text{odds increase with probability})$$

$$\iff z = \log \frac{p}{1-p} \geq \log \frac{t}{1-t} \quad (\text{take logs}).$$

# Changing the threshold moves the decision boundary

**1. Choose class 1 if  $p \geq t$ , where  $0 < t < 1$  and  $p = \sigma(z)$ .**

$$p \geq t \iff \frac{p}{1-p} \geq \frac{t}{1-t} \quad (\text{odds increase with probability})$$

$$\iff z = \log \frac{p}{1-p} \geq \log \frac{t}{1-t} \quad (\text{take logs}).$$

**2. Interpret:** lowering  $t$  expands the positive region. False negatives cannot increase; false positives cannot decrease on a fixed dataset. Choose  $t$  using validation data, not the final test set.

## Check: where is the boundary?

---

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?

## Check: where is the boundary?

---

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?
- 2 At  $x = 3$ , what are the score, probability, and predicted label?

## Check: where is the boundary?

---

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?
- 2 At  $x = 3$ , what are the score, probability, and predicted label?
- 3 Does a sigmoid make the boundary nonlinear in the input?

## Check: where is the boundary?

---

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?
- 2 At  $x = 3$ , what are the score, probability, and predicted label?
- 3 Does a sigmoid make the boundary nonlinear in the input?

Boundary:  $2x - 4 = 0$ , so  $x = 2$ . At  $x = 3$ :  $z = 2$ ,  $p \approx 0.881$ ,  $\hat{y} = 1$ . The boundary is still linear in  $x$ .

## Check: where is the boundary?

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?
- 2 At  $x = 3$ , what are the score, probability, and predicted label?
- 3 Does a sigmoid make the boundary nonlinear in the input?

Boundary:  $2x - 4 = 0$ , so  $x = 2$ . At  $x = 3$ :  $z = 2$ ,  $p \approx 0.881$ ,  $\hat{y} = 1$ . The boundary is still linear in  $x$ .

Now solve  $2x - 4 \geq \log \frac{t}{1-t}$  for other thresholds:

| Threshold $t$ | Score cutoff | Predict 1 if |
|---------------|--------------|--------------|
| 0.5           | 0            | $x \geq 2$   |

## Check: where is the boundary?

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?
- 2 At  $x = 3$ , what are the score, probability, and predicted label?
- 3 Does a sigmoid make the boundary nonlinear in the input?

Boundary:  $2x - 4 = 0$ , so  $x = 2$ . At  $x = 3$ :  $z = 2$ ,  $p \approx 0.881$ ,  $\hat{y} = 1$ . The boundary is still linear in  $x$ .

Now solve  $2x - 4 \geq \log \frac{t}{1-t}$  for other thresholds:

| Threshold $t$ | Score cutoff | Predict 1 if   |
|---------------|--------------|----------------|
| 0.2           | $-\log 4$    | $x \geq 1.307$ |
| 0.5           | 0            | $x \geq 2$     |

## Check: where is the boundary?

Let  $z = 2x - 4$  and  $p = \sigma(z)$ . First try threshold  $t = 1/2$ .

- 1 At what  $x$  is  $p = 1/2$ ?
- 2 At  $x = 3$ , what are the score, probability, and predicted label?
- 3 Does a sigmoid make the boundary nonlinear in the input?

Boundary:  $2x - 4 = 0$ , so  $x = 2$ . At  $x = 3$ :  $z = 2$ ,  $p \approx 0.881$ ,  $\hat{y} = 1$ . The boundary is still linear in  $x$ .

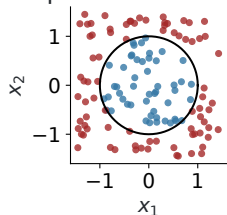
Now solve  $2x - 4 \geq \log \frac{t}{1-t}$  for other thresholds:

| Threshold $t$ | Score cutoff | Predict 1 if   |
|---------------|--------------|----------------|
| 0.2           | $-\log 4$    | $x \geq 1.307$ |
| 0.5           | 0            | $x \geq 2$     |
| 0.8           | $\log 4$     | $x \geq 2.693$ |

# Nonlinear features: a circle becomes a line

Class 1 lies outside the unit circle and class 0 lies inside.

Input space: circular boundary

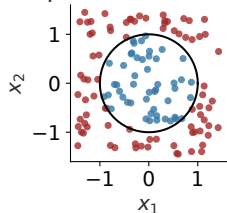


• Class 0      • Class 1

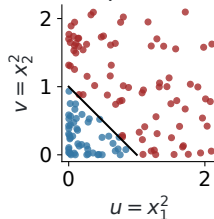
## Nonlinear features: a circle becomes a line

Class 1 lies outside the unit circle and class 0 lies inside. Apply the fixed map  $\phi(\mathbf{x}) = (x_1^2, x_2^2)^\top$ .

Input space: circular boundary



Feature space:  $u + v = 1$

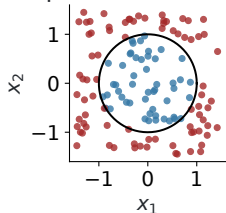


• Class 0    • Class 1

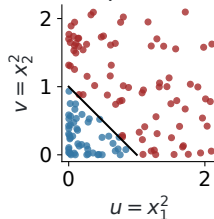
## Nonlinear features: a circle becomes a line

Class 1 lies outside the unit circle and class 0 lies inside. Apply the fixed map  $\phi(\mathbf{x}) = (x_1^2, x_2^2)^\top$ .

Input space: circular boundary



Feature space:  $u + v = 1$



• Class 0    • Class 1

Substitute  $u = x_1^2$ ,  $v = x_2^2$ :  $z = u + v - 1$ , with  $p = \sigma(z)$ .  
At threshold  $1/2$ , set  $z = 0$ : a line in  $(u, v)$ , a circle in  $(x_1, x_2)$ .

## More expressive features; the same learning problem

---

A full quadratic feature map in two inputs is

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^\top.$$

## More expressive features; the same learning problem

---

A full quadratic feature map in two inputs is

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^\top.$$

Its score and decision boundary are

$$z = b + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2, \quad z = 0.$$

## More expressive features; the same learning problem

---

A full quadratic feature map in two inputs is

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^\top.$$

Its score and decision boundary are

$$z = b + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2, \quad z = 0.$$

- The score is nonlinear in the original inputs but linear in the learned weights.

## More expressive features; the same learning problem

---

A full quadratic feature map in two inputs is

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^\top.$$

Its score and decision boundary are

$$z = b + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2, \quad z = 0.$$

- The score is nonlinear in the original inputs but linear in the learned weights.
- For a **fixed** feature map, cross-entropy remains convex in  $(\mathbf{w}, b)$ .

## More expressive features; the same learning problem

A full quadratic feature map in two inputs is

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^\top.$$

Its score and decision boundary are

$$z = b + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2, \quad z = 0.$$

- The score is nonlinear in the original inputs but linear in the learned weights.
- For a **fixed** feature map, cross-entropy remains convex in  $(\mathbf{w}, b)$ .
- More features increase flexibility, parameter count, and the opportunity to overfit.

Choose the feature degree and regularization with validation data. Later we will compare this with learning the feature map itself.

# Table of Contents

---

- 1 Review and Continuation
  - Review: regularization and model selection
  - Continue: interpreting bias and variance
- 2 From Regression to Classification
- 3 Logistic Regression: Model, Loss, Update**
  - A probability model
  - Learning with cross-entropy**
  - Optimization and regularization

## Choose parameters that make the observed labels likely

---

For sample  $n$ , let  $z_n = \mathbf{x}_n^\top \mathbf{w} + b$  and  $p_n = \sigma(z_n)$ .

1 **Start with the two labels:**

$$p(y_n \mid \mathbf{x}_n; \mathbf{w}, b) = \begin{cases} p_n, & y_n = 1, \\ 1 - p_n, & y_n = 0. \end{cases}$$

## Choose parameters that make the observed labels likely

---

For sample  $n$ , let  $z_n = \mathbf{x}_n^\top \mathbf{w} + b$  and  $p_n = \sigma(z_n)$ .

1 **Start with the two labels:**

$$p(y_n \mid \mathbf{x}_n; \mathbf{w}, b) = \begin{cases} p_n, & y_n = 1, \\ 1 - p_n, & y_n = 0. \end{cases}$$

2 **Write both cases in one expression:**  $p_n^{y_n} (1 - p_n)^{1-y_n}$ .

The exponent  $y_n$  selects  $p_n$ ;  $1 - y_n$  selects  $1 - p_n$ .

# Choose parameters that make the observed labels likely

---

For sample  $n$ , let  $z_n = \mathbf{x}_n^\top \mathbf{w} + b$  and  $p_n = \sigma(z_n)$ .

**1 Start with the two labels:**

$$p(y_n \mid \mathbf{x}_n; \mathbf{w}, b) = \begin{cases} p_n, & y_n = 1, \\ 1 - p_n, & y_n = 0. \end{cases}$$

**2 Write both cases in one expression:  $p_n^{y_n} (1 - p_n)^{1-y_n}$ .**

The exponent  $y_n$  selects  $p_n$ ;  $1 - y_n$  selects  $1 - p_n$ .

**3 Multiply over independent examples:**

$$\mathcal{L}(\mathbf{w}, b) = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}.$$

# Choose parameters that make the observed labels likely

For sample  $n$ , let  $z_n = \mathbf{x}_n^\top \mathbf{w} + b$  and  $p_n = \sigma(z_n)$ .

**1 Start with the two labels:**

$$p(y_n \mid \mathbf{x}_n; \mathbf{w}, b) = \begin{cases} p_n, & y_n = 1, \\ 1 - p_n, & y_n = 0. \end{cases}$$

**2 Write both cases in one expression:  $p_n^{y_n}(1 - p_n)^{1-y_n}$ .**

The exponent  $y_n$  selects  $p_n$ ;  $1 - y_n$  selects  $1 - p_n$ .

**3 Multiply over independent examples:**

$$\mathcal{L}(\mathbf{w}, b) = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}.$$

We model  $Y \mid X$  directly; no model for the input distribution is required.

# From a product of probabilities to a sum of losses

---

Start with  $\mathcal{L} = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}$ .

# From a product of probabilities to a sum of losses

---

Start with  $\mathcal{L} = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}$ .

$$\log \mathcal{L} = \sum_{n=1}^N \log [p_n^{y_n} (1 - p_n)^{1-y_n}] \quad (\text{product to sum})$$

# From a product of probabilities to a sum of losses

---

Start with  $\mathcal{L} = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}$ .

$$\begin{aligned}\log \mathcal{L} &= \sum_{n=1}^N \log [p_n^{y_n} (1 - p_n)^{1-y_n}] \quad (\text{product to sum}) \\ &= \sum_{n=1}^N [y_n \log p_n + (1 - y_n) \log(1 - p_n)] \quad (\text{log powers})\end{aligned}$$

## From a product of probabilities to a sum of losses

---

Start with  $\mathcal{L} = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}$ .

$$\begin{aligned}\log \mathcal{L} &= \sum_{n=1}^N \log [p_n^{y_n} (1 - p_n)^{1-y_n}] && \text{(product to sum)} \\ &= \sum_{n=1}^N [y_n \log p_n + (1 - y_n) \log(1 - p_n)] && \text{(log powers)} \\ J &= -\frac{1}{N} \log \mathcal{L} && \text{(negate and average).}\end{aligned}$$

Log is increasing, so maximizing  $\mathcal{L}$  and minimizing  $J$  select the same solutions.

# From a product of probabilities to a sum of losses

Start with  $\mathcal{L} = \prod_{n=1}^N p_n^{y_n} (1 - p_n)^{1-y_n}$ .

$$\begin{aligned}\log \mathcal{L} &= \sum_{n=1}^N \log [p_n^{y_n} (1 - p_n)^{1-y_n}] \quad (\text{product to sum}) \\ &= \sum_{n=1}^N [y_n \log p_n + (1 - y_n) \log(1 - p_n)] \quad (\text{log powers}) \\ J &= -\frac{1}{N} \log \mathcal{L} \quad (\text{negate and average}).\end{aligned}$$

Log is increasing, so maximizing  $\mathcal{L}$  and minimizing  $J$  select the same solutions.

Check: observed labels (1, 0), predictions (0.8, 0.3).

$$\mathcal{L} = 0.8 \times (1 - 0.3) = 0.56; J = -\frac{1}{2} \log 0.56 \approx 0.290.$$

## From probability loss to score loss: the algebra

---

Let  $p = \sigma(z) = e^z / (1 + e^z)$ . Use  $\log p = z - \log(1 + e^z)$  and  $\log(1 - p) = -\log(1 + e^z)$ .

$$\ell(z, y) = -y \log p - (1 - y) \log(1 - p)$$

**Start:** the loss in probability form.

## From probability loss to score loss: the algebra

---

Let  $p = \sigma(z) = e^z / (1 + e^z)$ . Use  $\log p = z - \log(1 + e^z)$  and  $\log(1 - p) = -\log(1 + e^z)$ .

$$\begin{aligned}\ell(z, y) &= -y \log p - (1 - y) \log(1 - p) \\ &= -y[z - \log(1 + e^z)] + (1 - y) \log(1 + e^z)\end{aligned}$$

**Substitute:** replace the two log probabilities using the identities above.

## From probability loss to score loss: the algebra

---

Let  $p = \sigma(z) = e^z / (1 + e^z)$ . Use  $\log p = z - \log(1 + e^z)$  and  $\log(1 - p) = -\log(1 + e^z)$ .

$$\begin{aligned}\ell(z, y) &= -y \log p - (1 - y) \log(1 - p) \\ &= -y [z - \log(1 + e^z)] + (1 - y) \log(1 + e^z) \\ &= \underbrace{[y + (1 - y)]}_1 \log(1 + e^z) - yz\end{aligned}$$

**Collect:** combine the two coefficients of  $\log(1 + e^z)$ .

## From probability loss to score loss: the algebra

---

Let  $p = \sigma(z) = e^z / (1 + e^z)$ . Use  $\log p = z - \log(1 + e^z)$  and  $\log(1 - p) = -\log(1 + e^z)$ .

$$\begin{aligned}\ell(z, y) &= -y \log p - (1 - y) \log(1 - p) \\ &= -y[z - \log(1 + e^z)] + (1 - y) \log(1 + e^z) \\ &= \underbrace{[y + (1 - y)]}_1 \log(1 + e^z) - yz \\ &= \log(1 + e^z) - yz.\end{aligned}$$

**Simplify:**  $y + (1 - y) = 1$ , so only the linear term depends on  $y$ .

## From probability loss to score loss: the algebra

Let  $p = \sigma(z) = e^z / (1 + e^z)$ . Use  $\log p = z - \log(1 + e^z)$  and  $\log(1 - p) = -\log(1 + e^z)$ .

$$\begin{aligned}\ell(z, y) &= -y \log p - (1 - y) \log(1 - p) \\ &= -y [z - \log(1 + e^z)] + (1 - y) \log(1 + e^z) \\ &= \underbrace{[y + (1 - y)]}_1 \log(1 + e^z) - yz \\ &= \log(1 + e^z) - yz.\end{aligned}$$

**Simplify:**  $y + (1 - y) = 1$ , so only the linear term depends on  $y$ .

Check:  $y = 0$  gives  $\log(1 + e^z)$ ;  $y = 1$  gives  $\log(1 + e^{-z})$ . Each penalizes a confidently wrong score.

## Negative log-likelihood gives binary cross-entropy

---

Maximizing  $\mathcal{L}$  is equivalent to minimizing the average negative log-likelihood:

$$\begin{aligned} J(\mathbf{w}, b) &= -\frac{1}{N} \log \mathcal{L}(\mathbf{w}, b) \\ &= \frac{1}{N} \sum_{n=1}^N \underbrace{[-y_n \log p_n - (1 - y_n) \log(1 - p_n)]}_{\text{binary cross-entropy}}. \end{aligned}$$

## Negative log-likelihood gives binary cross-entropy

---

Maximizing  $\mathcal{L}$  is equivalent to minimizing the average negative log-likelihood:

$$\begin{aligned} J(\mathbf{w}, b) &= -\frac{1}{N} \log \mathcal{L}(\mathbf{w}, b) \\ &= \frac{1}{N} \sum_{n=1}^N \underbrace{[-y_n \log p_n - (1 - y_n) \log(1 - p_n)]}_{\text{binary cross-entropy}}. \end{aligned}$$

Substituting  $p_n = \sigma(z_n)$  gives the equivalent score-based form:

$$J(\mathbf{w}, b) = \frac{1}{N} \sum_{n=1}^N [\log(1 + e^{z_n}) - y_n z_n].$$

## Negative log-likelihood gives binary cross-entropy

Maximizing  $\mathcal{L}$  is equivalent to minimizing the average negative log-likelihood:

$$\begin{aligned} J(\mathbf{w}, b) &= -\frac{1}{N} \log \mathcal{L}(\mathbf{w}, b) \\ &= \frac{1}{N} \sum_{n=1}^N \underbrace{[-y_n \log p_n - (1 - y_n) \log(1 - p_n)]}_{\text{binary cross-entropy}}. \end{aligned}$$

Substituting  $p_n = \sigma(z_n)$  gives the equivalent score-based form:

$$J(\mathbf{w}, b) = \frac{1}{N} \sum_{n=1}^N [\log(1 + e^{z_n}) - y_n z_n].$$

This is the loss we optimize. Classification error is a separate evaluation metric.

## Cross-entropy notices confidence, not just the label

---

For an observation with  $y = 1$ , the loss is  $-\log p$  (natural logarithm).

| Predicted $p$ | Predicted label | 0-1 loss | Cross-entropy |
|---------------|-----------------|----------|---------------|
| 0.9           | 1               | 0        | 0.105         |

---

## Cross-entropy notices confidence, not just the label

---

For an observation with  $y = 1$ , the loss is  $-\log p$  (natural logarithm).

| Predicted $p$ | Predicted label | 0-1 loss | Cross-entropy |
|---------------|-----------------|----------|---------------|
| 0.9           | 1               | 0        | 0.105         |
| 0.6           | 1               | 0        | 0.511         |

## Cross-entropy notices confidence, not just the label

For an observation with  $y = 1$ , the loss is  $-\log p$  (natural logarithm).

| Predicted $p$ | Predicted label | 0–1 loss | Cross-entropy |
|---------------|-----------------|----------|---------------|
| 0.9           | 1               | 0        | 0.105         |
| 0.6           | 1               | 0        | 0.511         |
| 0.1           | 0               | 1        | 2.303         |

## Cross-entropy notices confidence, not just the label

For an observation with  $y = 1$ , the loss is  $-\log p$  (natural logarithm).

| Predicted $p$ | Predicted label | 0-1 loss | Cross-entropy |
|---------------|-----------------|----------|---------------|
| 0.9           | 1               | 0        | 0.105         |
| 0.6           | 1               | 0        | 0.511         |
| 0.1           | 0               | 1        | 2.303         |

- Two correct classifications can have different training losses.

## Cross-entropy notices confidence, not just the label

For an observation with  $y = 1$ , the loss is  $-\log p$  (natural logarithm).

| Predicted $p$ | Predicted label | 0–1 loss | Cross-entropy |
|---------------|-----------------|----------|---------------|
| 0.9           | 1               | 0        | 0.105         |
| 0.6           | 1               | 0        | 0.511         |
| 0.1           | 0               | 1        | 2.303         |

- Two correct classifications can have different training losses.
- A confidently wrong prediction receives a large penalty.

## Cross-entropy notices confidence, not just the label

For an observation with  $y = 1$ , the loss is  $-\log p$  (natural logarithm).

| Predicted $p$ | Predicted label | 0–1 loss | Cross-entropy |
|---------------|-----------------|----------|---------------|
| 0.9           | 1               | 0        | 0.105         |
| 0.6           | 1               | 0        | 0.511         |
| 0.1           | 0               | 1        | 2.303         |

- Two correct classifications can have different training losses.
- A confidently wrong prediction receives a large penalty.
- For  $y = 0$ , the loss is  $-\log(1 - p)$  instead.

# Table of Contents

---

- 1 Review and Continuation
  - Review: regularization and model selection
  - Continue: interpreting bias and variance
- 2 From Regression to Classification
- 3 Logistic Regression: Model, Loss, Update**
  - A probability model
  - Learning with cross-entropy
  - Optimization and regularization

# The gradient starts with a simple residual

---

For one example,  $\ell(z, y) = \log(1 + e^z) - yz$  and  $z = \mathbf{x}^\top \mathbf{w} + b$ .

1 **Differentiate with respect to the score:**

$$\frac{\partial \ell}{\partial z} = \frac{e^z}{1 + e^z} - y = \sigma(z) - y = p - y.$$

# The gradient starts with a simple residual

---

For one example,  $\ell(z, y) = \log(1 + e^z) - yz$  and  $z = \mathbf{x}^\top \mathbf{w} + b$ .

- 1 **Differentiate with respect to the score:**

$$\frac{\partial \ell}{\partial z} = \frac{e^z}{1 + e^z} - y = \sigma(z) - y = p - y.$$

- 2 **Differentiate the score:**  $\nabla_{\mathbf{w}} z = \mathbf{x}$ ,  $\partial z / \partial b = 1$ .

# The gradient starts with a simple residual

---

For one example,  $\ell(z, y) = \log(1 + e^z) - yz$  and  $z = \mathbf{x}^\top \mathbf{w} + b$ .

- 1 **Differentiate with respect to the score:**

$$\frac{\partial \ell}{\partial z} = \frac{e^z}{1 + e^z} - y = \sigma(z) - y = p - y.$$

- 2 **Differentiate the score:**  $\nabla_{\mathbf{w}} z = \mathbf{x}$ ,  $\partial z / \partial b = 1$ .

- 3 **Multiply by the chain rule:**

$$\nabla_{\mathbf{w}} \ell = \frac{\partial \ell}{\partial z} \nabla_{\mathbf{w}} z = (p - y) \mathbf{x}, \quad \frac{\partial \ell}{\partial b} = (p - y) \cdot 1.$$

# The gradient starts with a simple residual

For one example,  $\ell(z, y) = \log(1 + e^z) - yz$  and  $z = \mathbf{x}^\top \mathbf{w} + b$ .

**1 Differentiate with respect to the score:**

$$\frac{\partial \ell}{\partial z} = \frac{e^z}{1 + e^z} - y = \sigma(z) - y = p - y.$$

**2 Differentiate the score:  $\nabla_{\mathbf{w}} z = \mathbf{x}$ ,  $\partial z / \partial b = 1$ .**

**3 Multiply by the chain rule:**

$$\nabla_{\mathbf{w}} \ell = \frac{\partial \ell}{\partial z} \nabla_{\mathbf{w}} z = (p - y) \mathbf{x}, \quad \frac{\partial \ell}{\partial b} = (p - y) \cdot 1.$$

If  $y = 1$  but  $p$  is too small,  $p - y < 0$ : a gradient step increases this example's score. As in least squares, the gradient is a residual times the input.

# From one example to a training algorithm

---

Let  $X \in \mathbb{R}^{N \times d}$  have rows  $\mathbf{x}_n^\top$  and  $\mathbf{p} = \sigma(X\mathbf{w} + b\mathbf{1})$  elementwise.

$$\nabla_{\mathbf{w}} J = \frac{1}{N} \sum_{n=1}^N (p_n - y_n) \mathbf{x}_n \quad (\text{average the sample gradients})$$

# From one example to a training algorithm

---

Let  $X \in \mathbb{R}^{N \times d}$  have rows  $\mathbf{x}_n^\top$  and  $\mathbf{p} = \sigma(X\mathbf{w} + b\mathbf{1})$  elementwise.

$$\begin{aligned}\nabla_{\mathbf{w}} J &= \frac{1}{N} \sum_{n=1}^N (p_n - y_n) \mathbf{x}_n && \text{(average the sample gradients)} \\ &= \frac{1}{N} X^\top (\mathbf{p} - \mathbf{y}), && \frac{\partial J}{\partial b} = \frac{1}{N} \sum_n (p_n - y_n).\end{aligned}$$

# From one example to a training algorithm

---

Let  $X \in \mathbb{R}^{N \times d}$  have rows  $\mathbf{x}_n^\top$  and  $\mathbf{p} = \sigma(X\mathbf{w} + b\mathbf{1})$  elementwise.

$$\begin{aligned}\nabla_{\mathbf{w}} J &= \frac{1}{N} \sum_{n=1}^N (p_n - y_n) \mathbf{x}_n && \text{(average the sample gradients)} \\ &= \frac{1}{N} X^\top (\mathbf{p} - \mathbf{y}), && \frac{\partial J}{\partial b} = \frac{1}{N} \sum_n (p_n - y_n).\end{aligned}$$

- 1 Compute scores and probabilities on the training batch.

# From one example to a training algorithm

---

Let  $X \in \mathbb{R}^{N \times d}$  have rows  $\mathbf{x}_n^\top$  and  $\mathbf{p} = \sigma(X\mathbf{w} + b\mathbf{1})$  elementwise.

$$\begin{aligned}\nabla_{\mathbf{w}} J &= \frac{1}{N} \sum_{n=1}^N (p_n - y_n) \mathbf{x}_n && \text{(average the sample gradients)} \\ &= \frac{1}{N} X^\top (\mathbf{p} - \mathbf{y}), && \frac{\partial J}{\partial b} = \frac{1}{N} \sum_n (p_n - y_n).\end{aligned}$$

- 1 Compute scores and probabilities on the training batch.
- 2 Compute residuals  $p_n - y_n$  and average the gradients.

# From one example to a training algorithm

---

Let  $X \in \mathbb{R}^{N \times d}$  have rows  $\mathbf{x}_n^\top$  and  $\mathbf{p} = \sigma(X\mathbf{w} + b\mathbf{1})$  elementwise.

$$\begin{aligned}\nabla_{\mathbf{w}} J &= \frac{1}{N} \sum_{n=1}^N (p_n - y_n) \mathbf{x}_n && \text{(average the sample gradients)} \\ &= \frac{1}{N} X^\top (\mathbf{p} - \mathbf{y}), && \frac{\partial J}{\partial b} = \frac{1}{N} \sum_n (p_n - y_n).\end{aligned}$$

- 1 Compute scores and probabilities on the training batch.
- 2 Compute residuals  $p_n - y_n$  and average the gradients.
- 3 Update with learning rate  $\eta > 0$ :  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} J$ ,  $b \leftarrow b - \eta \partial J / \partial b$ .

# From one example to a training algorithm

Let  $X \in \mathbb{R}^{N \times d}$  have rows  $\mathbf{x}_n^\top$  and  $\mathbf{p} = \sigma(X\mathbf{w} + b\mathbf{1})$  elementwise.

$$\begin{aligned}\nabla_{\mathbf{w}} J &= \frac{1}{N} \sum_{n=1}^N (p_n - y_n) \mathbf{x}_n && \text{(average the sample gradients)} \\ &= \frac{1}{N} X^\top (\mathbf{p} - \mathbf{y}), && \frac{\partial J}{\partial b} = \frac{1}{N} \sum_n (p_n - y_n).\end{aligned}$$

- 1 Compute scores and probabilities on the training batch.
- 2 Compute residuals  $p_n - y_n$  and average the gradients.
- 3 Update with learning rate  $\eta > 0$ :  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} J$ ,  $b \leftarrow b - \eta \partial J / \partial b$ .

The stationary equation has no general closed-form solution; repeat these steps.

## One gradient step by hand

---

Given  $x = 2$ ,  $y = 1$ ,  $w = b = 0$ , and learning rate  $\eta = 0.1$ :

**Try first:** should the probability of class 1 increase or decrease?

# One gradient step by hand

---

Given  $x = 2$ ,  $y = 1$ ,  $w = b = 0$ , and learning rate  $\eta = 0.1$ :

1 **Predict:**  $z = wx + b = 0$ , so  $p = \sigma(0) = 0.5$ .

# One gradient step by hand

---

Given  $x = 2$ ,  $y = 1$ ,  $w = b = 0$ , and learning rate  $\eta = 0.1$ :

- 1 **Predict:**  $z = wx + b = 0$ , so  $p = \sigma(0) = 0.5$ .
- 2 **Compute the residual:**  $r = p - y = 0.5 - 1 = -0.5$ .  
Hence  $\partial\ell/\partial w = rx = -1$  and  $\partial\ell/\partial b = r = -0.5$ .

# One gradient step by hand

---

Given  $x = 2$ ,  $y = 1$ ,  $w = b = 0$ , and learning rate  $\eta = 0.1$ :

- 1 **Predict:**  $z = wx + b = 0$ , so  $p = \sigma(0) = 0.5$ .
- 2 **Compute the residual:**  $r = p - y = 0.5 - 1 = -0.5$ .  
Hence  $\partial\ell/\partial w = rx = -1$  and  $\partial\ell/\partial b = r = -0.5$ .
- 3 **Subtract the gradient:**  $w_{\text{new}} = 0 - 0.1(-1) = 0.1$ ,  $b_{\text{new}} = 0 - 0.1(-0.5) = 0.05$ .

# One gradient step by hand

---

Given  $x = 2$ ,  $y = 1$ ,  $w = b = 0$ , and learning rate  $\eta = 0.1$ :

- 1 **Predict:**  $z = wx + b = 0$ , so  $p = \sigma(0) = 0.5$ .
  - 2 **Compute the residual:**  $r = p - y = 0.5 - 1 = -0.5$ .  
Hence  $\partial \ell / \partial w = rx = -1$  and  $\partial \ell / \partial b = r = -0.5$ .
  - 3 **Subtract the gradient:**  $w_{\text{new}} = 0 - 0.1(-1) = 0.1$ ,  $b_{\text{new}} = 0 - 0.1(-0.5) = 0.05$ .
  - 4 **Predict again:**  $z_{\text{new}} = 0.1 \cdot 2 + 0.05 = 0.25$ , so  $p_{\text{new}} \approx 0.562$ .
- Check:**  $-\log 0.5 = 0.693$  decreases to  $-\log 0.562 \approx 0.576$ .

## Convexity helps, but does not guarantee a finite minimizer

---

For  $\ell(z, y) = \log(1 + e^z) - yz$ ,

$$\frac{\partial^2 \ell}{\partial z^2} = \sigma(z)(1 - \sigma(z)) \geq 0.$$

An affine score and averaging preserve convexity, so  $J(\mathbf{w}, b)$  is convex.

## Convexity helps, but does not guarantee a finite minimizer

---

For  $\ell(z, y) = \log(1 + e^z) - yz$ ,

$$\frac{\partial^2 \ell}{\partial z^2} = \sigma(z)(1 - \sigma(z)) \geq 0.$$

An affine score and averaging preserve convexity, so  $J(\mathbf{w}, b)$  is convex.

- Any attained local minimum is global; convexity alone does not imply uniqueness.

## Convexity helps, but does not guarantee a finite minimizer

---

For  $\ell(z, y) = \log(1 + e^z) - yz$ ,

$$\frac{\partial^2 \ell}{\partial z^2} = \sigma(z)(1 - \sigma(z)) \geq 0.$$

An affine score and averaging preserve convexity, so  $J(\mathbf{w}, b)$  is convex.

- Any attained local minimum is global; convexity alone does not imply uniqueness.
- On strictly separable data, scaling a separating score can keep decreasing the loss.

## Convexity helps, but does not guarantee a finite minimizer

---

For  $\ell(z, y) = \log(1 + e^z) - yz$ ,

$$\frac{\partial^2 \ell}{\partial z^2} = \sigma(z)(1 - \sigma(z)) \geq 0.$$

An affine score and averaging preserve convexity, so  $J(\mathbf{w}, b)$  is convex.

- Any attained local minimum is global; convexity alone does not imply uniqueness.
- On strictly separable data, scaling a separating score can keep decreasing the loss.
- The unregularized loss can approach zero with parameters growing without bound.

## Convexity helps, but does not guarantee a finite minimizer

For  $\ell(z, y) = \log(1 + e^z) - yz$ ,

$$\frac{\partial^2 \ell}{\partial z^2} = \sigma(z)(1 - \sigma(z)) \geq 0.$$

An affine score and averaging preserve convexity, so  $J(\mathbf{w}, b)$  is convex.

- Any attained local minimum is global; convexity alone does not imply uniqueness.
- On strictly separable data, scaling a separating score can keep decreasing the loss.
- The unregularized loss can approach zero with parameters growing without bound.

Regularization controls coefficient growth and connects back to model choice.

# From the gradient to the Hessian

---

Include the intercept:  $\tilde{\mathbf{x}}_n = (\mathbf{x}_n^\top, 1)^\top$ ,  $\boldsymbol{\theta} = (\mathbf{w}^\top, b)^\top$ , and  $p_n = \sigma(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})$ .

1 **Start from the gradient:**  $\nabla J = \frac{1}{N} \sum_n (p_n - y_n) \tilde{\mathbf{x}}_n$ .

# From the gradient to the Hessian

---

Include the intercept:  $\tilde{\mathbf{x}}_n = (\mathbf{x}_n^\top, 1)^\top$ ,  $\boldsymbol{\theta} = (\mathbf{w}^\top, b)^\top$ , and  $p_n = \sigma(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})$ .

1 **Start from the gradient:**  $\nabla J = \frac{1}{N} \sum_n (p_n - y_n) \tilde{\mathbf{x}}_n$ .

2 **Only  $p_n$  depends on the parameters:**

$$\nabla_{\boldsymbol{\theta}} p_n = \underbrace{\sigma'(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})}_{p_n(1-p_n)} \tilde{\mathbf{x}}_n.$$

# From the gradient to the Hessian

---

Include the intercept:  $\tilde{\mathbf{x}}_n = (\mathbf{x}_n^\top, 1)^\top$ ,  $\boldsymbol{\theta} = (\mathbf{w}^\top, b)^\top$ , and  $p_n = \sigma(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})$ .

1 **Start from the gradient:**  $\nabla J = \frac{1}{N} \sum_n (p_n - y_n) \tilde{\mathbf{x}}_n$ .

2 **Only  $p_n$  depends on the parameters:**

$$\nabla_{\boldsymbol{\theta}} p_n = \underbrace{\sigma'(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})}_{p_n(1-p_n)} \tilde{\mathbf{x}}_n.$$

3 **Differentiate once more and collect terms:**

$$H := \nabla^2 J = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top = \frac{1}{N} \tilde{X}^\top D \tilde{X}.$$

# From the gradient to the Hessian

Include the intercept:  $\tilde{\mathbf{x}}_n = (\mathbf{x}_n^\top, 1)^\top$ ,  $\boldsymbol{\theta} = (\mathbf{w}^\top, b)^\top$ , and  $p_n = \sigma(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})$ .

1 **Start from the gradient:**  $\nabla J = \frac{1}{N} \sum_n (p_n - y_n) \tilde{\mathbf{x}}_n$ .

2 **Only  $p_n$  depends on the parameters:**

$$\nabla_{\boldsymbol{\theta}} p_n = \underbrace{\sigma'(\tilde{\mathbf{x}}_n^\top \boldsymbol{\theta})}_{p_n(1-p_n)} \tilde{\mathbf{x}}_n.$$

3 **Differentiate once more and collect terms:**

$$H := \nabla^2 J = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top = \frac{1}{N} \tilde{X}^\top D \tilde{X}.$$

$D = \text{diag}(p_n(1 - p_n))$ . Each example contributes a nonnegative weight times an outer product. Next: show that  $H \succeq 0$ .

## Why convex? The Hessian is positive semidefinite

---

Recall  $H = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top$ . To prove  $H \succeq 0$ , take an **arbitrary vector**  $v$  and check  $v^\top H v$ .

## Why convex? The Hessian is positive semidefinite

---

Recall  $H = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top$ . To prove  $H \succeq 0$ , take an **arbitrary vector**  $v$  and check  $v^\top H v$ .

$$v^\top H v = \frac{1}{N} \sum_n p_n(1 - p_n) v^\top \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top v$$

## Why convex? The Hessian is positive semidefinite

---

Recall  $H = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top$ . To prove  $H \succeq 0$ , take an **arbitrary vector**  $v$  and check  $v^\top H v$ .

$$\begin{aligned} v^\top H v &= \frac{1}{N} \sum_n p_n(1 - p_n) v^\top \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top v \\ &= \frac{1}{N} \sum_n \underbrace{p_n(1 - p_n)}_{\geq 0} \underbrace{(\tilde{\mathbf{x}}_n^\top v)^2}_{\geq 0} \geq 0. \end{aligned}$$

The scalar  $v^\top \tilde{\mathbf{x}}_n$  equals  $\tilde{\mathbf{x}}_n^\top v$ , so their product is a square.

## Why convex? The Hessian is positive semidefinite

---

Recall  $H = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top$ . To prove  $H \succeq 0$ , take an **arbitrary vector**  $v$  and check  $v^\top H v$ .

$$\begin{aligned} v^\top H v &= \frac{1}{N} \sum_n p_n(1 - p_n) v^\top \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top v \\ &= \frac{1}{N} \sum_n \underbrace{p_n(1 - p_n)}_{\geq 0} \underbrace{(\tilde{\mathbf{x}}_n^\top v)^2}_{\geq 0} \geq 0. \end{aligned}$$

The scalar  $v^\top \tilde{\mathbf{x}}_n$  equals  $\tilde{\mathbf{x}}_n^\top v$ , so their product is a square.

- Therefore  $H \succeq 0$  and  $J$  is convex jointly in weights and intercept.

## Why convex? The Hessian is positive semidefinite

---

Recall  $H = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top$ . To prove  $H \succeq 0$ , take an **arbitrary vector**  $v$  and check  $v^\top H v$ .

$$\begin{aligned} v^\top H v &= \frac{1}{N} \sum_n p_n(1 - p_n) v^\top \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top v \\ &= \frac{1}{N} \sum_n \underbrace{p_n(1 - p_n)}_{\geq 0} \underbrace{(\tilde{\mathbf{x}}_n^\top v)^2}_{\geq 0} \geq 0. \end{aligned}$$

The scalar  $v^\top \tilde{\mathbf{x}}_n$  equals  $\tilde{\mathbf{x}}_n^\top v$ , so their product is a square.

- Therefore  $H \succeq 0$  and  $J$  is convex jointly in weights and intercept.
- Full column rank of  $\tilde{X}$  gives strict convexity at finite parameters.

## Why convex? The Hessian is positive semidefinite

Recall  $H = \frac{1}{N} \sum_n p_n(1 - p_n) \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top$ . To prove  $H \succeq 0$ , take an **arbitrary vector**  $v$  and check  $v^\top H v$ .

$$\begin{aligned} v^\top H v &= \frac{1}{N} \sum_n p_n(1 - p_n) v^\top \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top v \\ &= \frac{1}{N} \sum_n \underbrace{p_n(1 - p_n)}_{\geq 0} \underbrace{(\tilde{\mathbf{x}}_n^\top v)^2}_{\geq 0} \geq 0. \end{aligned}$$

The scalar  $v^\top \tilde{\mathbf{x}}_n$  equals  $\tilde{\mathbf{x}}_n^\top v$ , so their product is a square.

- Therefore  $H \succeq 0$  and  $J$  is convex jointly in weights and intercept.
- Full column rank of  $\tilde{X}$  gives strict convexity at finite parameters.
- A finite minimizer need not exist: that is a separate question.

## Perfect classification can still leave no finite minimizer

---

Two examples:  $(x, y) = (-1, 0)$  and  $(1, 1)$ . Use  $z = wx$ ,  $b = 0$ .

---

|     |                         |                       |        |
|-----|-------------------------|-----------------------|--------|
| $w$ | $\Pr(Y = 1 \mid x = 1)$ | <b>Training error</b> | $J(w)$ |
|-----|-------------------------|-----------------------|--------|

---

---

## Perfect classification can still leave no finite minimizer

Two examples:  $(x, y) = (-1, 0)$  and  $(1, 1)$ . Use  $z = wx$ ,  $b = 0$ . For any  $w > 0$ , both labels are classified correctly.

$$\ell(-w, 0) = \ell(w, 1) = \log(1 + e^{-w}), \quad J(w) = \log(1 + e^{-w}).$$

| $w$ | $\Pr(Y = 1 \mid x = 1)$ | Training error | $J(w)$ |
|-----|-------------------------|----------------|--------|
|     |                         |                |        |
|     |                         |                |        |
|     |                         |                |        |
|     |                         |                |        |

## Perfect classification can still leave no finite minimizer

Two examples:  $(x, y) = (-1, 0)$  and  $(1, 1)$ . Use  $z = wx$ ,  $b = 0$ . For any  $w > 0$ , both labels are classified correctly.

$$\ell(-w, 0) = \ell(w, 1) = \log(1 + e^{-w}), \quad J(w) = \log(1 + e^{-w}).$$

| $w$ | $\Pr(Y = 1 \mid x = 1)$ | Training error | $J(w)$ |
|-----|-------------------------|----------------|--------|
| 1   | 0.731                   | 0              | 0.313  |

## Perfect classification can still leave no finite minimizer

Two examples:  $(x, y) = (-1, 0)$  and  $(1, 1)$ . Use  $z = wx$ ,  $b = 0$ . For any  $w > 0$ , both labels are classified correctly.

$$\ell(-w, 0) = \ell(w, 1) = \log(1 + e^{-w}), \quad J(w) = \log(1 + e^{-w}).$$

| $w$ | $\Pr(Y = 1 \mid x = 1)$ | <b>Training error</b> | $J(w)$    |
|-----|-------------------------|-----------------------|-----------|
| 1   | 0.731                   | 0                     | 0.313     |
| 5   | 0.993                   | 0                     | 0.00672   |
| 10  | 0.99995                 | 0                     | 0.0000454 |

## Perfect classification can still leave no finite minimizer

Two examples:  $(x, y) = (-1, 0)$  and  $(1, 1)$ . Use  $z = wx$ ,  $b = 0$ . For any  $w > 0$ , both labels are classified correctly.

$$\ell(-w, 0) = \ell(w, 1) = \log(1 + e^{-w}), \quad J(w) = \log(1 + e^{-w}).$$

| $w$ | $\Pr(Y = 1 \mid x = 1)$ | Training error | $J(w)$    |
|-----|-------------------------|----------------|-----------|
| 1   | 0.731                   | 0              | 0.313     |
| 5   | 0.993                   | 0              | 0.00672   |
| 10  | 0.99995                 | 0              | 0.0000454 |

Differentiate:  $J'(w) = -1/(1 + e^w) < 0$  for every finite  $w$ . So increasing  $w$  always lowers the loss;  $J(w) \rightarrow 0$  only as  $w \rightarrow \infty$ .

# Regularized logistic regression: the same workflow returns

---

**1. Add a penalty:** weights are penalized; the intercept is not.

$$J_{\lambda}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

# Regularized logistic regression: the same workflow returns

---

**1. Add a penalty:** weights are penalized; the intercept is not.

$$J_{\lambda}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

**2. Differentiate the new term:**  $\nabla_{\mathbf{w}}(\frac{\lambda}{2} \sum_j w_j^2) = \lambda \mathbf{w}$ .

$$\nabla_{\mathbf{w}} J_{\lambda} = \frac{1}{N} X^{\top} (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}.$$

# Regularized logistic regression: the same workflow returns

---

**1. Add a penalty:** weights are penalized; the intercept is not.

$$J_{\lambda}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

**2. Differentiate the new term:**  $\nabla_{\mathbf{w}}(\frac{\lambda}{2} \sum_j w_j^2) = \lambda \mathbf{w}$ .

$$\nabla_{\mathbf{w}} J_{\lambda} = \frac{1}{N} X^{\top} (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}.$$

- Fit feature scaling on training data; reuse it for validation and test data.

# Regularized logistic regression: the same workflow returns

---

**1. Add a penalty:** weights are penalized; the intercept is not.

$$J_{\lambda}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

**2. Differentiate the new term:**  $\nabla_{\mathbf{w}}(\frac{\lambda}{2} \sum_j w_j^2) = \lambda \mathbf{w}$ .

$$\nabla_{\mathbf{w}} J_{\lambda} = \frac{1}{N} X^{\top} (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}.$$

- Fit feature scaling on training data; reuse it for validation and test data.
- Fit  $(\mathbf{w}, b)$  on training data for each candidate  $\lambda$ .

# Regularized logistic regression: the same workflow returns

---

**1. Add a penalty:** weights are penalized; the intercept is not.

$$J_{\lambda}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

**2. Differentiate the new term:**  $\nabla_{\mathbf{w}}(\frac{\lambda}{2} \sum_j w_j^2) = \lambda \mathbf{w}$ .

$$\nabla_{\mathbf{w}} J_{\lambda} = \frac{1}{N} X^{\top} (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}.$$

- Fit feature scaling on training data; reuse it for validation and test data.
- Fit  $(\mathbf{w}, b)$  on training data for each candidate  $\lambda$ .
- Select  $\lambda$  on validation data; evaluate once on untouched test data.

# Regularized logistic regression: the same workflow returns

---

**1. Add a penalty:** weights are penalized; the intercept is not.

$$J_{\lambda}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad \lambda > 0.$$

**2. Differentiate the new term:**  $\nabla_{\mathbf{w}}(\frac{\lambda}{2} \sum_j w_j^2) = \lambda \mathbf{w}$ .

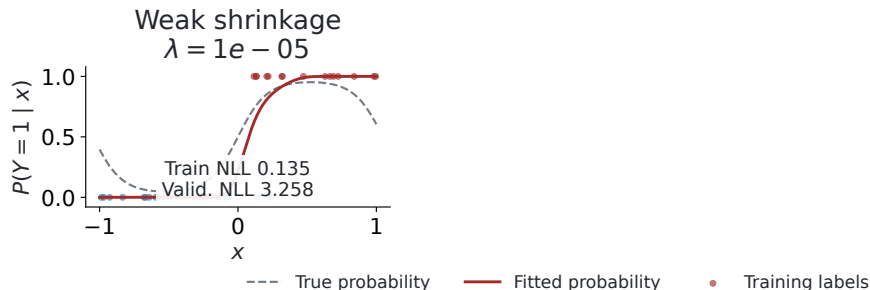
$$\nabla_{\mathbf{w}} J_{\lambda} = \frac{1}{N} X^{\top} (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}.$$

- Fit feature scaling on training data; reuse it for validation and test data.
- Fit  $(\mathbf{w}, b)$  on training data for each candidate  $\lambda$ .
- Select  $\lambda$  on validation data; evaluate once on untouched test data.

Stronger shrinkage can reduce sensitivity, but excessive shrinkage can underfit.

# Same features and data; different regularization strengths

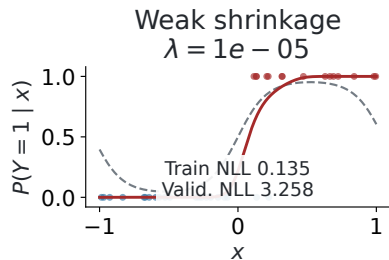
Synthetic example: 36 training samples, 400 validation samples; degree-9 features standardized using training statistics only.



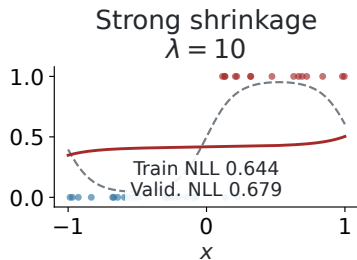
**1. Weak shrinkage:** training NLL is low. Compare it with validation NLL.

## Same features and data; different regularization strengths

Synthetic example: 36 training samples, 400 validation samples; degree-9 features standardized using training statistics only.



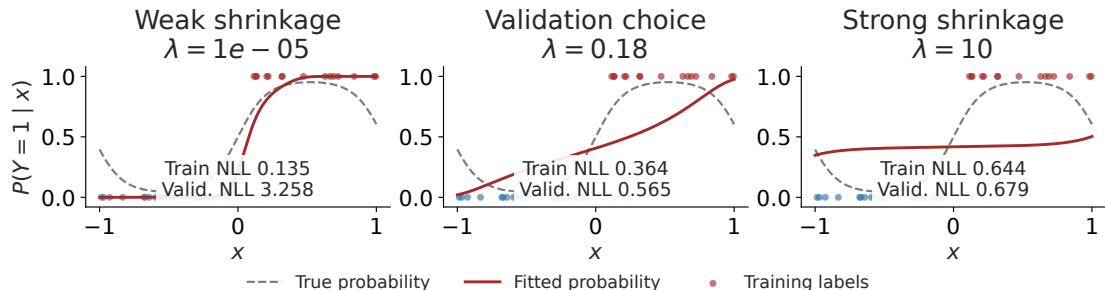
--- True probability    — Fitted probability    • Training labels



**2. Strong shrinkage:** the fitted probability becomes almost flat. What pattern is lost?

## Same features and data; different regularization strengths

Synthetic example: 36 training samples, 400 validation samples; degree-9 features standardized using training statistics only.

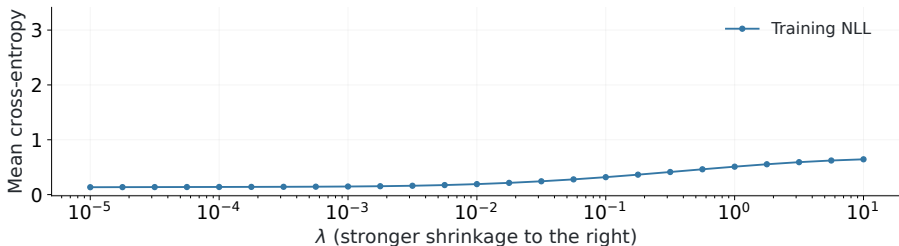


**3. Validation choice:** an intermediate penalty gives the lowest validation NLL of the candidates.

The dashed truth is known only because this is a synthetic experiment.

# Choose regularization by held-out performance

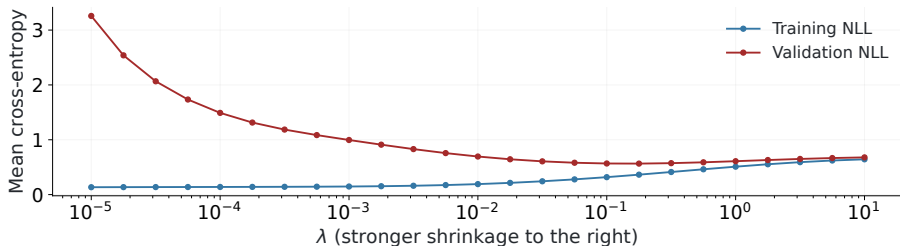
Same experiment, 25 candidate  $\lambda$  values; all models use the same training set.



**1. Look at training loss alone:** it favors weak regularization. Would you trust this choice?

# Choose regularization by held-out performance

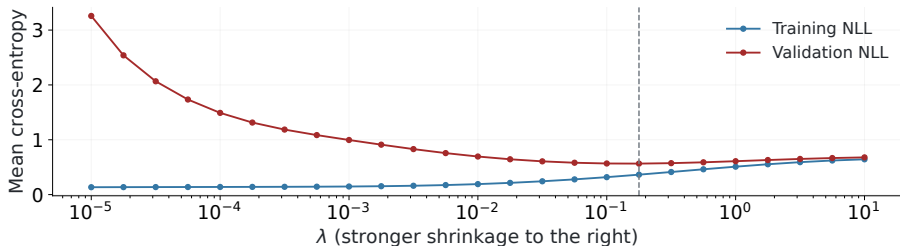
Same experiment, 25 candidate  $\lambda$  values; all models use the same training set.



**2. Add independent validation:** weak regularization performs poorly on held-out labels.

# Choose regularization by held-out performance

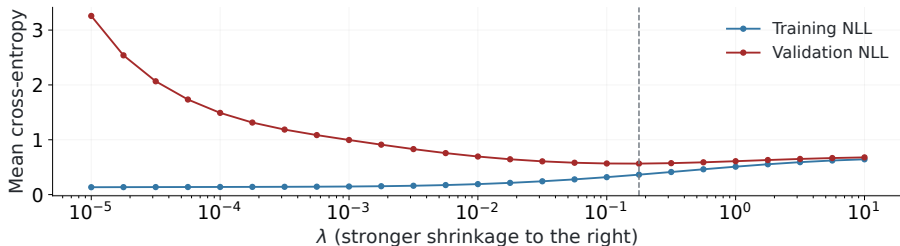
Same experiment, 25 candidate  $\lambda$  values; all models use the same training set.



**3. Select the validation minimum:**  $\lambda \approx 0.178$  among these 25 candidates.

# Choose regularization by held-out performance

Same experiment, 25 candidate  $\lambda$  values; all models use the same training set.



**3. Select the validation minimum:**  $\lambda \approx 0.178$  among these 25 candidates.

Both curves measure **unpenalized mean cross-entropy** after fitting. The best validation value is a selection result; final evaluation still needs untouched test data.

## Exit check: connect the whole pipeline

---

- 1 In the bias–variance formula, what changes across repetitions?

## Exit check: connect the whole pipeline

---

- 1 In the bias–variance formula, what changes across repetitions?
- 2 Does a nonlinear sigmoid imply a nonlinear decision boundary?

## Exit check: connect the whole pipeline

---

- 1 In the bias–variance formula, what changes across repetitions?
- 2 Does a nonlinear sigmoid imply a nonlinear decision boundary?
- 3 Why optimize cross-entropy rather than 0–1 error directly?

## Exit check: connect the whole pipeline

---

- 1 In the bias–variance formula, what changes across repetitions?
- 2 Does a nonlinear sigmoid imply a nonlinear decision boundary?
- 3 Why optimize cross-entropy rather than 0–1 error directly?
- 4 Which data choose  $\lambda$ ? Which data estimate final performance?

## Exit check: connect the whole pipeline

---

- 1 In the bias–variance formula, what changes across repetitions?
- 2 Does a nonlinear sigmoid imply a nonlinear decision boundary?
- 3 Why optimize cross-entropy rather than 0–1 error directly?
- 4 Which data choose  $\lambda$ ? Which data estimate final performance?

Training sets; no (with linear features); a smooth likelihood-based objective; validation data choose  $\lambda$ , and untouched test data evaluate the result.

## Exit check: connect the whole pipeline

---

- 1 In the bias–variance formula, what changes across repetitions?
- 2 Does a nonlinear sigmoid imply a nonlinear decision boundary?
- 3 Why optimize cross-entropy rather than 0–1 error directly?
- 4 Which data choose  $\lambda$ ? Which data estimate final performance?

Training sets; no (with linear features); a smooth likelihood-based objective; validation data choose  $\lambda$ , and untouched test data evaluate the result.

**Next lecture:** continue from logistic regression toward generalized linear models.

# Table of Contents

---

## 4 Optional Extensions

## Optional extensions: use as needed

---

- MAP: a probabilistic interpretation of ridge regularization.
- Double descent: why the classical U-shape is not universal.
- Maximum margin: a preview of support vector machines.
- Bayes optimality: a supporting proof.

The core lecture ends at the exit check.

## Optional: why a Gaussian prior gives ridge

---

MAP (maximum a posteriori) maximizes the posterior density. Use a Gaussian linear model and prior (centered form):

$$y_n \mid \mathbf{x}_n, \mathbf{w} \sim \mathcal{N}(\mathbf{x}_n^\top \mathbf{w}, \sigma^2), \quad \mathbf{w} \sim \mathcal{N}(0, \tau^2 I).$$

## Optional: why a Gaussian prior gives ridge

---

MAP (maximum a posteriori) maximizes the posterior density. Use a Gaussian linear model and prior (centered form):

$$y_n \mid \mathbf{x}_n, \mathbf{w} \sim \mathcal{N}(\mathbf{x}_n^\top \mathbf{w}, \sigma^2), \quad \mathbf{w} \sim \mathcal{N}(0, \tau^2 I).$$

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg \min_{\mathbf{w}} \{ -\log p(\mathbf{y} \mid X, \mathbf{w}) - \log p(\mathbf{w}) \}$$

## Optional: why a Gaussian prior gives ridge

---

MAP (maximum a posteriori) maximizes the posterior density. Use a Gaussian linear model and prior (centered form):

$$y_n \mid \mathbf{x}_n, \mathbf{w} \sim \mathcal{N}(\mathbf{x}_n^\top \mathbf{w}, \sigma^2), \quad \mathbf{w} \sim \mathcal{N}(0, \tau^2 I).$$

$$\begin{aligned} \hat{\mathbf{w}}_{\text{MAP}} &= \arg \min_{\mathbf{w}} \{ -\log p(\mathbf{y} \mid X, \mathbf{w}) - \log p(\mathbf{w}) \} \\ &= \arg \min_{\mathbf{w}} \left\{ \frac{\|\mathbf{y} - X\mathbf{w}\|_2^2}{2\sigma^2} + \frac{\|\mathbf{w}\|_2^2}{2\tau^2} \right\} \end{aligned}$$

## Optional: why a Gaussian prior gives ridge

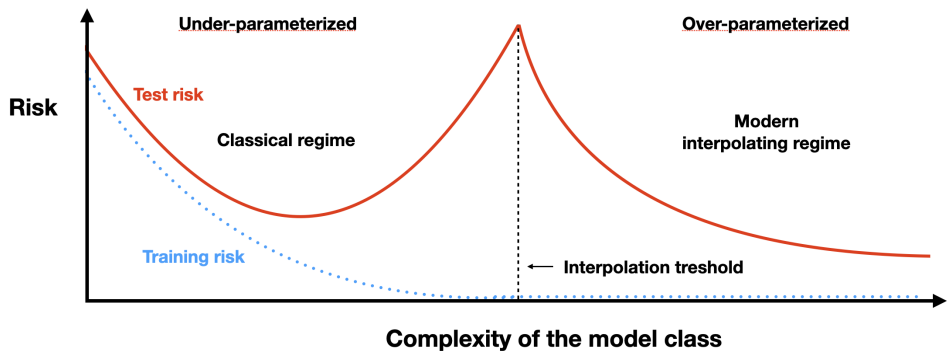
MAP (maximum a posteriori) maximizes the posterior density. Use a Gaussian linear model and prior (centered form):

$$y_n \mid \mathbf{x}_n, \mathbf{w} \sim \mathcal{N}(\mathbf{x}_n^\top \mathbf{w}, \sigma^2), \quad \mathbf{w} \sim \mathcal{N}(0, \tau^2 I).$$

$$\begin{aligned} \hat{\mathbf{w}}_{\text{MAP}} &= \arg \min_{\mathbf{w}} \{ -\log p(\mathbf{y} \mid X, \mathbf{w}) - \log p(\mathbf{w}) \} \\ &= \arg \min_{\mathbf{w}} \left\{ \frac{\|\mathbf{y} - X\mathbf{w}\|_2^2}{2\sigma^2} + \frac{\|\mathbf{w}\|_2^2}{2\tau^2} \right\} \\ &= \arg \min_{\mathbf{w}} \left\{ \frac{\|\mathbf{y} - X\mathbf{w}\|_2^2}{2N} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \right\}, \quad \lambda = \frac{\sigma^2}{N\tau^2}. \end{aligned}$$

A narrower prior (smaller  $\tau^2$ ) means stronger shrinkage. With a Bernoulli likelihood, the same prior gives L2-penalized logistic regression.

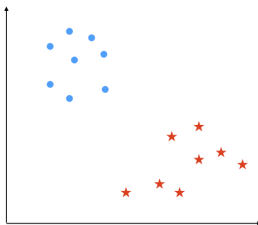
## Optional: Double descent



# Separating hyperplane

---

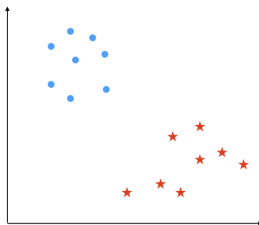
Assume we restrict ourselves to linear decision boundaries (hyperplane):



# Separating hyperplane

---

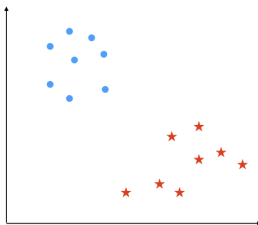
Assume we restrict ourselves to linear decision boundaries (hyperplane):



Assume the data are linearly separable, i.e., a separating hyperplane exists

# Separating hyperplane

Assume we restrict ourselves to linear decision boundaries (hyperplane):



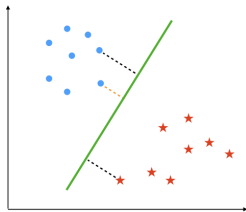
Assume the data are linearly separable, i.e., a separating hyperplane exists

Which separating hyperplane would you pick?

# Margin

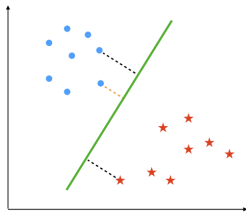
---

**Key concept:** The margin is the distance from the hyperplane to the closest point.



# Margin

**Key concept:** The margin is the distance from the hyperplane to the closest point.

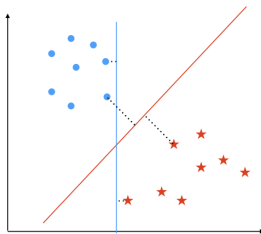


Take the one with the largest margin!

# Max-margin separating hyperplane

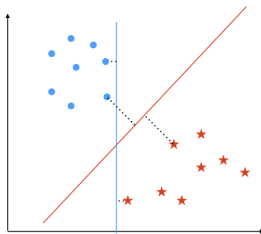
---

Choose the hyperplane which maximizes the margin



# Max-margin separating hyperplane

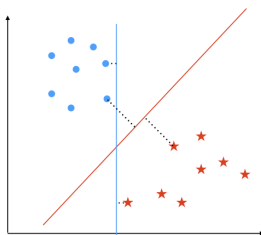
Choose the hyperplane which maximizes the margin



**WHY?**

# Max-margin separating hyperplane

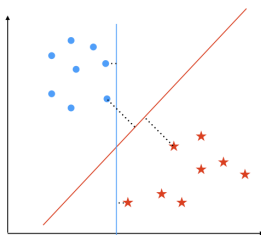
Choose the hyperplane which maximizes the margin



**WHY?** A larger geometric margin tolerates larger input perturbations before a label changes

# Max-margin separating hyperplane

Choose the hyperplane which maximizes the margin



**WHY?** A larger geometric margin tolerates larger input perturbations before a label changes

⇒ This motivates support vector machines (SVMs); logistic regression optimizes a different objective.

## Optional: why the Bayes rule is optimal

---

For any classifier  $g : \mathcal{X} \rightarrow \{0, 1\}$ ,

$$\begin{aligned} R(g) &= \Pr(Y \neq g(X)) \\ &= \mathbb{E}_X [\Pr(Y \neq g(X) \mid X)] \\ &\geq \mathbb{E}_X [\min\{q(X), 1 - q(X)\}] = R(f^*). \end{aligned}$$

The threshold rule  $f^*(\mathbf{x}) = \mathbf{1}\{q(\mathbf{x}) \geq 1/2\}$  attains the pointwise minimum.

Bayes error can be positive even with perfect knowledge of  $q$ . The rule above assumes equal costs for the two kinds of error.

# Reading materials & Reference

---

## Reading materials:

- Chapter 3, Stanford CS 229 Lecture Notes, [https://cs229.stanford.edu/notes2022fall/main\\_notes.pdf](https://cs229.stanford.edu/notes2022fall/main_notes.pdf)
- Lecture 4, Stanford CS 231n, <http://cs231n.stanford.edu/schedule.html>

## Reference:

- EPFL, CS-433 Machine Learning, [https://github.com/epfml/ML\\_course](https://github.com/epfml/ML_course)